Github

inky4832@daum.net

원격 저장소 생성 및 관리

1. Github 개요

Github 용도

git 원격 저장소를 제공하는 대표적인 서비스가 Github이다. 단순히 원격 저장소만을 제공하는 것이 아니라 여러가지 프로젝트 진행을 원활하게 하는 도구를 함께 제공한다.

주요 기능

cloud에 git 저장소 제공 (push/pull , public/private)
code 관리 및 협업개발 가능 (issue, projects, Pull Request(PR))
automations & CI/CD (Github Actions 이용)

Github 장점

전 세계에서 진행되는 오픈 소스 프로젝트가 많이 모여 있어서 이에 참여하고 오픈소스에 기여할 수 있는 기회가 제공된다.

개발자는 Github를 이용해서 자신이 작성했던 코드 그 자체를 곧바로 제공할 수 있다.

IT 개발과 관련된 많은 디자이너 및 기획자 역시 자신이 준비했던 문서 및 포트폴리오를 공개할 수 있다. 개발 협업이 가능하다.

2. 기본 명령어

명령어	기능
git clone	원격 저장소의 모든 내용을 로컬 저장소로 복사한다.
git remote	로컬 저장소를 특정 원격 저장소와 연결한다.
git push	로컬 저장소의 커밋 내역을 원격 저장소로 보낸다.
git fetch	로컬 저장소와 원격 저장소의 커밋 버전이 다를 때, 원격 저장소의 커밋 내역을 로컬로 가져온다. 필요시 git merge 로 나중에 병합할 수 있다.
git pull	원격 저장소의 커밋 내역을 로컬로 가져와서 자동으로 병합한다.
git branch -r git branch -a	원격 브랜치 목록 보기 로컬 및 원격 브랜치 목록 보기
git push origin --delete <브랜치명>	원격 브랜치 삭제
git fetch -p origin	로컬의 원격 브랜치 정보와 실제 원격 브랜치 정보 동기화. 예> 원격에서 branch 삭제 되어 실제 원격 branch와 로컬에서 보는 원격 branch가 다를 때 등.

명령어: git remote

https://git-scm.com/docs/git-remote

NAME

git-remote - Manage set of tracked repositories

SYNOPSIS

```
git remote [-v | --verbose]
git remote add [-t <branch>] [-m <master>] [-f] [--[no-]tags] [--mirror=
(fetch|push)] <name> <URL>
git remote rename [--[no-]progress] <old> <new>
git remote remove <name>
git remote set-head <name> (-a | --auto | -d | --delete | <branch>)
git remote set-branches [--add] <name> <branch>…
git remote get-url [--push] [--all] <name>
git remote set-url [--push] <name> <newurl> [<oldurl>]
git remote set-url --add [--push] <name> <newurl>
git remote set-url --delete [--push] <name> <URL>
git remote [-v | --verbose] show [-n] <name>…
git remote prune [-n | --dry-run] <name>…
git remote [-v | --verbose] update [-p | --prune] [(<group> | <remote>)…]
```

DESCRIPTION

Manage the set of repositories ("remotes") whose branches you track.

명령어: git push

https://git-scm.com/docs/git-push

§ **NAME**

git-push - Update remote refs along with associated objects

SYNOPSIS

```
git push [--all | --branches | --mirror | --tags] [--follow-tags] [--
atomic] [-n | --dry-run] [--receive-pack=<git-receive-pack>]
           [--repo=<repository>] [-f | --force] [-d | --delete] [--prune]
[-q | --quiet] [-v | --verbose]
           [-u | --set-upstream] [-o <string> | --push-option=<string>]
           [--[no-]signed|--signed=(true|false|if-asked)]
           [--force-with-lease[=<refname>[:<expect>]] [--force-if-
includes]]
           [--no-verify] [<repository> [<refspec>…]]
```

DESCRIPTION

Updates remote refs using local refs, while sending objects necessary to complete the given refs.

You can make interesting things happen to a repository every time you push into it, by setting up hooks there. See documentation for git-receive-pack[1].

When the command line does not specify where to push with the `<repository>` argument, `branch.*.remote` configuration for the current branch is consulted to determine where to push. If the configuration is missing, it defaults to `origin`.

명령어: git fetch

https://git-scm.com/docs/git-fetch

NAME

git-fetch - Download objects and refs from another repository

SYNOPSIS

```
git fetch [<options>] [<repository> [<refspec>…]]
git fetch [<options>] <group>
git fetch --multiple [<options>] [(<repository> | <group>)…]
git fetch --all [<options>]
```

DESCRIPTION

Fetch branches and/or tags (collectively, "refs") from one or more other repositories, along with the objects necessary to complete their histories. Remote-tracking branches are updated (see the description of <refspec> below for ways to control this behavior).

By default, any tag that points into the histories being fetched is also f
point at branches that you are interested in. This default behavior can
no-tags options or by configuring remote.<name>.tagOpt. By using a
you can fetch tags that do not point into branches you are interested i

`git fetch` can fetch from either a single named repository or URL,
if <group> is given and there is a remotes.<group> entry in the config

When no remote is specified, by default the `origin` remote will be
branch configured for the current branch.

`-p`
`--prune`

Before fetching, remove any remote-tracking references that no longer exist on the remote. Tags are not subject to pruning if they are fetched only because of the default tag auto-following or due to a --tags option. However, if tags are fetched due to an explicit refspec (either on the command line or in the remote configuration, for example if the remote was cloned with the --mirror option), then they are also subject to pruning. Supplying `--prune-tags` is a shorthand for providing the tag refspec.

See the PRUNING section below for more details.

명령어: git pull

https://git-scm.com/docs/git-pull

NAME

git-pull - Fetch from and integrate with another repository or a local branch

SYNOPSIS

```
git pull [<options>] [<repository> [<refspec>…]]
```

DESCRIPTION

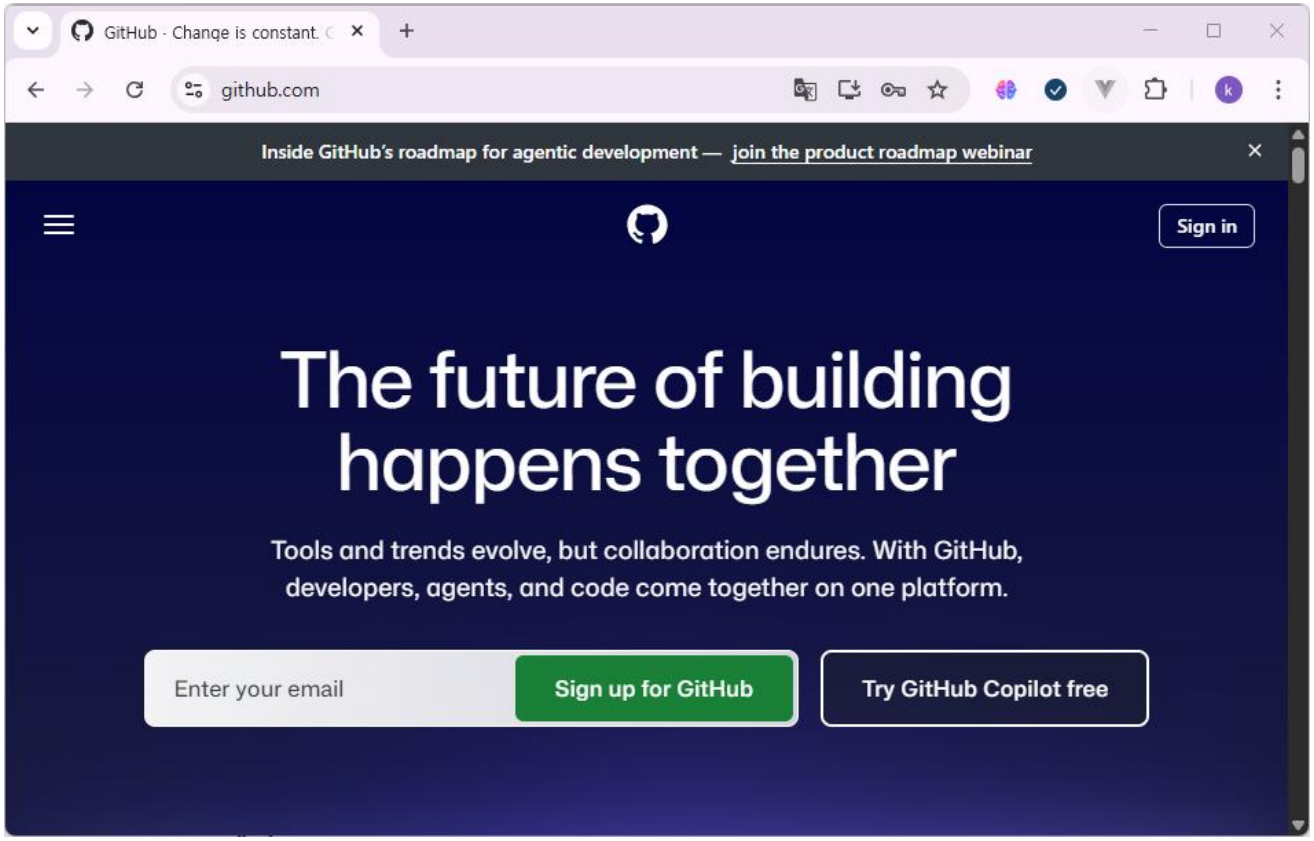
Incorporates changes from a remote repository into the current branch. If the current branch is behind the remote, then by default it will fast-forward the current branch to match the remote. If the current branch and the remote have diverged, the user needs to specify how to reconcile the divergent branches with `--rebase` or `--no-rebase` (or the corresponding configuration option in `pull.rebase`).

More precisely, `git pull` runs `git fetch` with the given parameters and then depending on configuration options or command line flags, will call either `git rebase` or `git merge` to reconcile diverging branches.

3. Github 가입

Github 가입 및 로그인

https://github.com/



4. 원격 저장소 생성 및 연결

가. 빈 원격 저장소 생성

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? Import a repository.
Required fields are marked with an asterisk (*).

General

Owner * / Repository name *

inky4832 / study **study**

study is available.

Great repository names are short and memorable. How about shiny-dollop?

Description

0 / 350 characters

Configuration

Choose visibility * Choose who can see and commit to this repository	Public
Add README READMEs can be used as longer descriptions. About READMEs	Off
Add .gitignore .gitignore tells git which files not to track. About ignoring files	No .gitignore
Add license Licenses explain how others can use your code. About licenses	No license

Create repository

Quick setup — if you've done this kind of thing before

Set up in Desktop or HTTPS SSH https://github.com/inky4832/study.git

Get started by creating a new file or uploading an existing file. We recommend every repository

로컬 저장소가 없는 경우

...or create a new repository on the command line

```
echo "# study" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/inky4832/study.git
git push -u origin main
```

로컬 저장소가 있는 경우

...or push an existing repository from the command line

```
git remote add origin https://github.com/inky4832/study.git
git branch -M main
git push -u origin main
```

4. 원격 저장소 생성 및 연결

나. 로컬 저장소와 빈 원격 저장소 연결

문법: git remote add origin https://~

로컬 저장소 생성

```
$ pwd
$ mkdir github_tutorial
$ cd github_tutorial/
$ git init
```

로컬 저장소와 원격 저장소 연결

```
$ git remote add origin https://github.com/inky4832/study.git
```

5. 원격 저장소 연결상태 확인 및 종료

원격 저장소 연결 상태 확인

문법: git remote
git remote -v

```
$ git remote -v
origin  https://github.com/inky4832/study.git (fetch)
origin  https://github.com/inky4832/study.git (push)
```

원격 저장소 연결 종료

문법: git remote remove origin
git remote rm origin

```
$ git remote rm origin
```

6. main 브랜치를 원격 저장소에 올리기

로컬 커밋 내역을 원격 저장소에 올리기

문법: git push origin main
git push origin --all

로컬 저장소 커밋

```
$ touch hello.txt
# hello.txt 내용 수정
$ cat hello.txt
  hello1
$ git add hello.txt
$ git commit -m “hello1 commit”
```

원격 저장소 연결 및 확인

```
$ git remote add origin https://github.com/inky4832/study.git
$ git remote -v
```

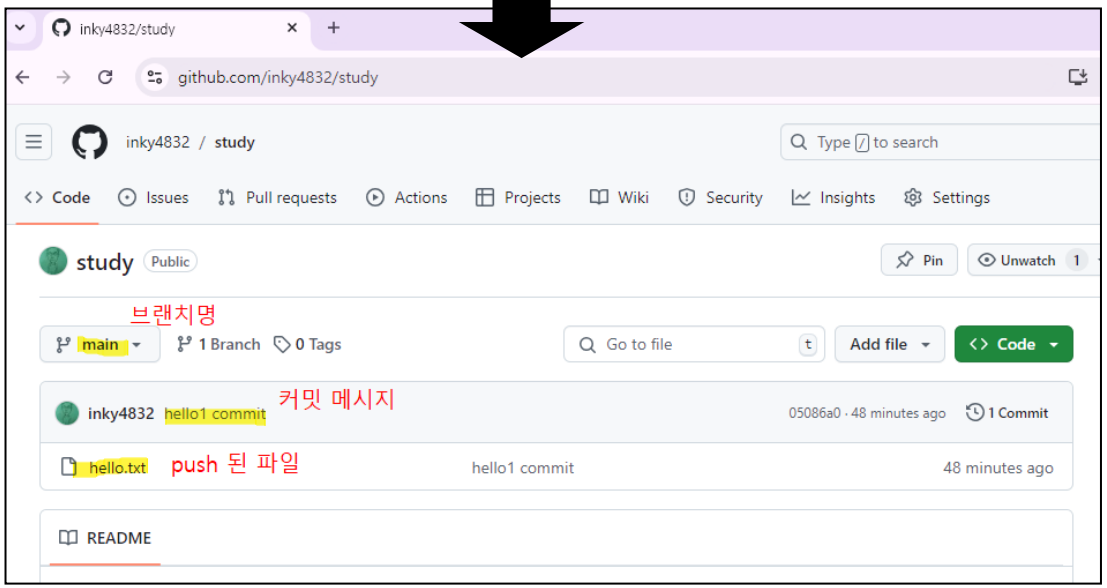
6. main 브랜치를 원격 저장소에 올리기

원격 저장소에 올리기

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (main)
$ git status
On branch main
nothing to commit, working tree clean
```

push 하기전에 반드시 status로 확인하여 아무런 변경사항이 없어야 된다. 있다면 commit 먼저 작업해야 된다.

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (main)
$ git push origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 212 bytes | 212.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/inky4832/study.git
 * [new branch]      main -> main
```



6. main 브랜치를 원격 저장소에 올리기

커밋 정보 보기

```
$ git log
commit 05086a08b8fc29f061e803251db388f573c62364 (HEAD -> main, origin/main)
Author: inky4832 <inky4832@daum.net>
Date:   Sat Sep 21 12:55:06 2024 +0900

    hello1 commit

inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (main)
$
```

로컬 원격

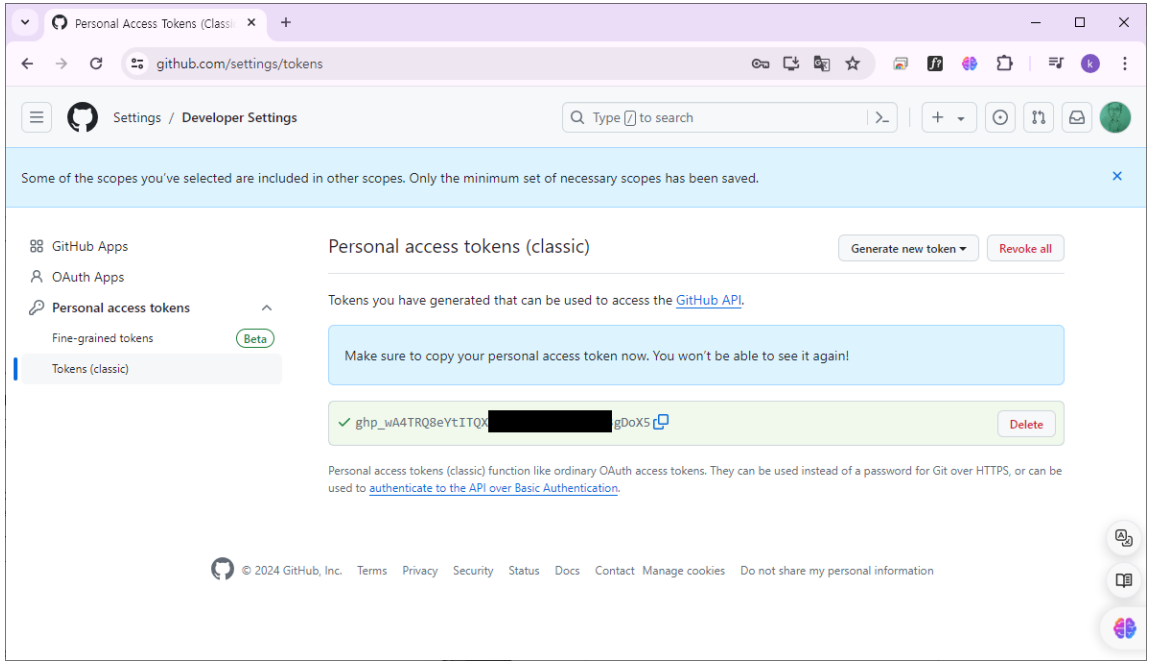
7. 원격 저장소에 올리기 (403 에러 발생된 경우)

원격 저장소에 올리기

```
$ git push origin main
remote: Permission to inky4832/study.git denied to kyung-yeol-in.
fatal: unable to access 'https://github.com/inky4832/study.git/':
The requested URL returned error: 403
```

token 얻기

https://github.com/settings/tokens



7. 원격 저장소에 올리기 (403 에러 발생된 경우)

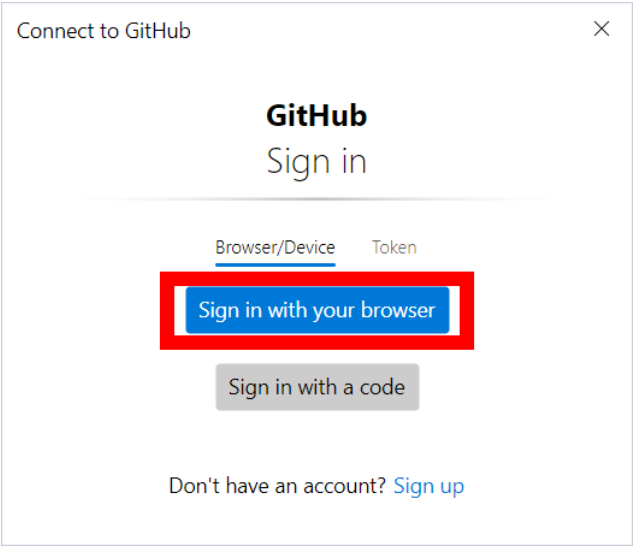
Token 이용한 권한 설정

문법: git remote set-url origin https://계정명@github.com/inky4832/study.git

```
$ git remote set-url origin https://inky4832@github.com/inky4832/study.git
```

원격 저장소에 다시 올리기

```
$ git push origin main
```




```
$ git push origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 211 bytes | 211.00
KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-
reused 0 (from 0)
To https://github.com/inky4832/study.git
 * [new branch]      main -> main
```

8. hotfix 브랜치를 원격 저장소에 올리기

로컬 커밋 내역을 원격 저장소에 올리기

문법: git push origin hotfix

hotfix 브랜치 생성 및 커밋

```
$ git branch hotfix
$ git switch hotfix
$ git branch
# hello.txt 수정
$ cat hello.txt
  hello1
  hello2 hotfix
$ git commit -am “hello2 hotfix commit”
```

8. hotfix 브랜치를 원격 저장소에 올리기

커밋 정보 보기

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (hotfix)
$ git log
commit 3c681a47f3c2813e31ca4940fb68978e666070a1 (HEAD -> hotfix)
Author: inky4832 <inky4832@daum.net>
Date:   Sat Sep 21 14:35:38 2024 +0900

    hello2 hotfix commit

commit 05086a08b8fc29f061e803251db388f573c62364 (origin/main, main)
Author: inky4832 <inky4832@daum.net>
Date:   Sat Sep 21 12:55:06 2024 +0900

    hello1 commit
```

```
$ git log main      로컬 저장소 main 브랜치 커밋 정보
commit 05086a08b8fc29f061e803251db388f573c62364 (origin/main, main)
Author: inky4832 <inky4832@daum.net>
Date:   Sat Sep 21 12:55:06 2024 +0900

    hello1 commit
```

```
$ git log origin/main    원격 저장소 main 브랜치 커밋 정보
commit 05086a08b8fc29f061e803251db388f573c62364 (origin/main, main)
Author: inky4832 <inky4832@daum.net>
Date:   Sat Sep 21 12:55:06 2024 +0900

    hello1 commit
```

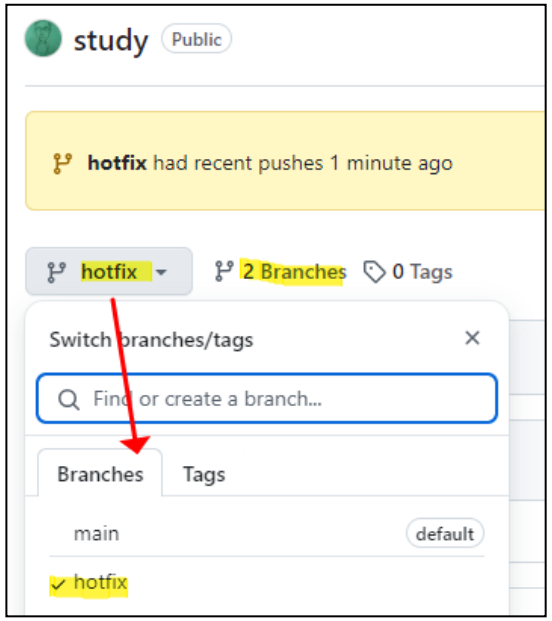
8. hotfix 브랜치를 원격 저장소에 올리기

원격 저장소 연결 및 확인

```
$ git remote add origin https://github.com/inky4832/study.git
$ git remote -v
```

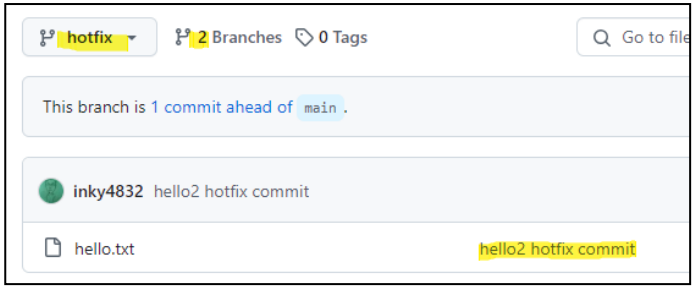
원격 저장소에 올리기

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (hotfix)
$ git push origin hotfix
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (3/3), 259 bytes | 259.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'hotfix' on GitHub by visiting:
remote:      https://github.com/inky4832/study/pull/new/hotfix
remote:
To https://github.com/inky4832/study.git
 * [new branch]      hotfix -> hotfix
```



원격 브랜치 목록보기

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (hotfix)
$ git branch -r
  origin/hotfix
  origin/main
```



9. hotfix 브랜치를 원격 저장소에서 삭제하기

원격 저장소 삭제하기

문법: git push origin --delete hotfix

원격 hotfix 브랜치 삭제

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (hotfix)
$ git branch -r
  origin/hotfix
  origin/main

inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (hotfix)
$ git push origin --delete hotfix
To https://github.com/inky4832/study.git
 - [deleted]         hotfix

inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (hotfix)
$ git branch -r
  origin/main
```

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (hotfix)
$ git branch -a
* hotfix
  main
  remotes/origin/main
```

9. hotfix 브랜치를 원격 저장소에서 삭제하기

원격 hotfix 브랜치 삭제후 작업

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (main)
$ git remote rm origin
```
명시적으로 원격 연결 종료

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (main)
$ git remote -v
```
원격 종료했기 때문에 출력 안됨

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (main)
$ git branch -r
```
원격 종료했기 때문에 출력 안됨

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (main)
$ git remote add origin https://github.com/inky4832/study.git
```
명시적으로 원격 연결

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (main)
$ git remote -v
origin  https://github.com/inky4832/study.git (fetch)
origin  https://github.com/inky4832/study.git (push)
```
원격 연결했기 때문에 출력됨

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (main)
$ git branch -r
```
원격 연결 되었으나 출력 안됨
실제 원격 브랜치 정보와 로컬이 알고 있는
원격 브랜치 정보가 다르기 때문이다.

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (main)
$ git fetch -p origin
From https://github.com/inky4832/study
 * [new branch]      main       -> origin/main
```
실제 원격 브랜치 정보와 로컬의 원격 브랜치 정보를
동기화 처리

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (main)
$ git branch -r
  origin/main
```
동기화 했기 때문에 정보 출력됨

fetch 와 pull

1. 원격 추적 브랜치 (Remote Tracking Branch)

개요

원격 저장소와 push/clone 작업등으로 연동이 되면 로컬에는 브랜치 참조가 2개가 생긴다. 하나는 표준 main 브랜치 참조이고 다른 하나는 origin/main 브랜치 참조이다. 이때 origin/main 브랜치 참조를 원격 추적 브랜치(Remote Tracking Branch)라고 한다. 이것은 마지막으로 원격 저장소와 통신한 시점을 기억하는 포인터 역할을 담당한다.

표준 main 브랜치 참조는 로컬에서 발생되는 가장 최신 커밋을 참조하고 origin/main 브랜치는 원격의 가장 최신 커밋을 참조한다.

따라서 처음에는 표준 main 브랜치 참조와 위치가 같지만 로컬에서 지속적으로 커밋하면 점점 표준 main 브랜치 참조와 멀어지게 된다.

로컬/원격 브랜치 보기

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (main)
$ git branch -a
  hotfix
* main  표준 main 브랜치
  remotes/origin/main  원격 추척 브랜치
```

1. 원격 추적 브랜치 (Remote Trackin Branch)

원격 브랜치 정보 확인

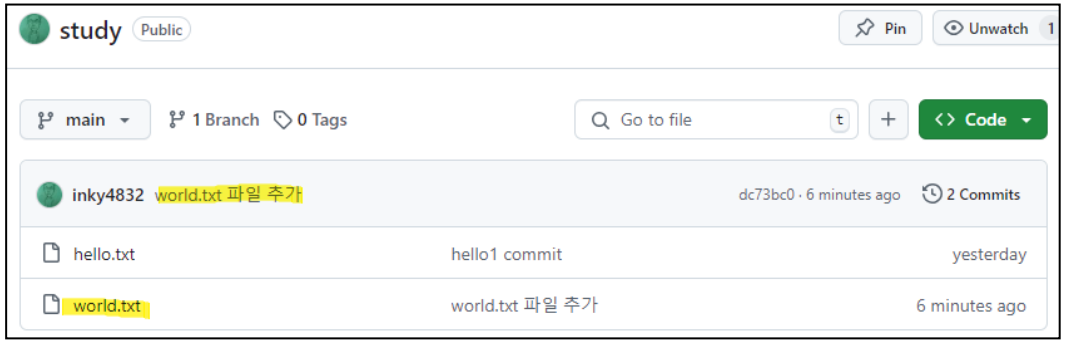
origin/main인 원격추적 참조를 통해서 로컬이 원격보다 얼마나 앞서 있는지 알 수 있다. 또한 origin/main 정보를 살펴 볼 수도 있다.

```
$ git switch main
# hello.txt 수정
$ cat hello.txt
  hello1
  hello2
$ git commit -am “hello2 commit”
$ git checkout origin/main
  .. You are in 'detached HEAD' state.
# hello.txt 확인    ( origin/main 정보 확인 )
$ git switch -     ( 이전 브랜치인 main으로 복귀)
```

2. fetch 이용한 원격 저장소에서 커밋 내역만 가져오기

문법: git fetch 후에 git merge

원격 저장소 변경 (world.txt 추가)



이 경우에 git push 하면 [rejected] 에러가 발생된다. 이유는 로컬저장소와 원격 저장소 간에 커밋 버전이 일치하지 않기 때문이다. 협업할 때 매우 자주 발생되는 에러이다.

```
$ git push origin main
To https://github.com/inky4832/study.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/inky4832/study.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

2. fetch 이용한 원격 저장소에서 커밋 내역만 가져오기

git fetch

git fetch 명령어는 원격 저장소의 커밋 내역만 가져온다.

```
$ git fetch
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 918 bytes | 65.00 KiB/s, done.
From https://github.com/inky4832/study
   2fbefc5..6ef2532  main       -> origin/main
```

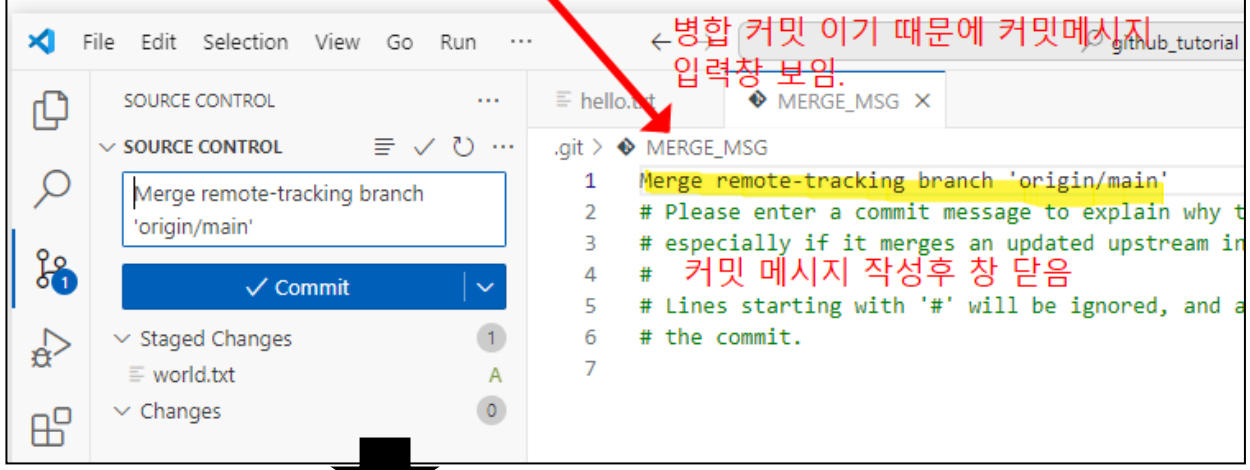
로컬 저장소와 원격 저장소의 변경 내역 비교

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (main)
$ git log main..origin/main
commit dc73bc0e76b53b3b996cf526e07a155449d57969 (origin/main)
Author: inky4832 <inky4832@daum.net>
Date:   Sun Sep 22 12:57:57 2024 +0900

    world.txt 파일 추가
```

2. fetch 이용한 원격 저장소에서 커밋 내역만 가져오기

로컬 저장소와 원격 저장소의 병합 (병합커밋)

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (main)
$ git merge origin/main  main
hint: Waiting for your editor to close the file...
```




```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (main)
$ git merge origin/main  main
Merge made by the 'ort' strategy.
 world.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 world.txt
```

병합 커밋

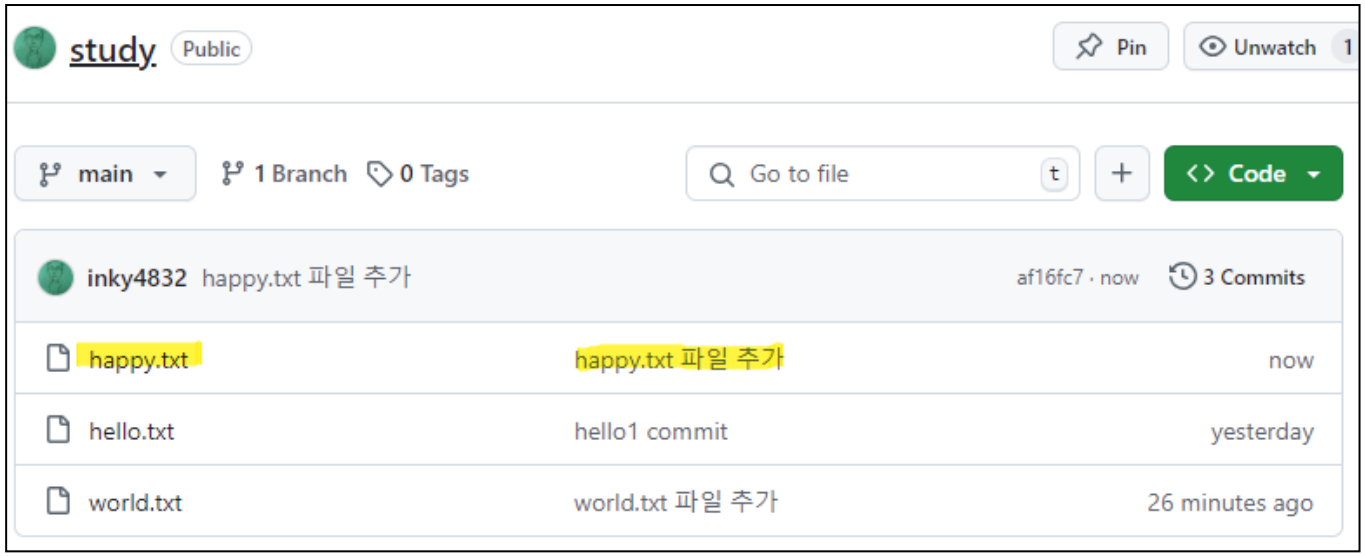
```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (main)
$ git log --oneline
f44b872 (HEAD -> main) Merge remote-tracking branch 'origin/main'
dc73bc0 (origin/main) world.txt 파 일  추 가
a2ad850 hello2 commit
05086a0 hello1 commit
```

3. pull 이용한 원격 저장소와 로컬 저장소 병합하기

문법: git pull origin main

git pull = git fetch + git merge

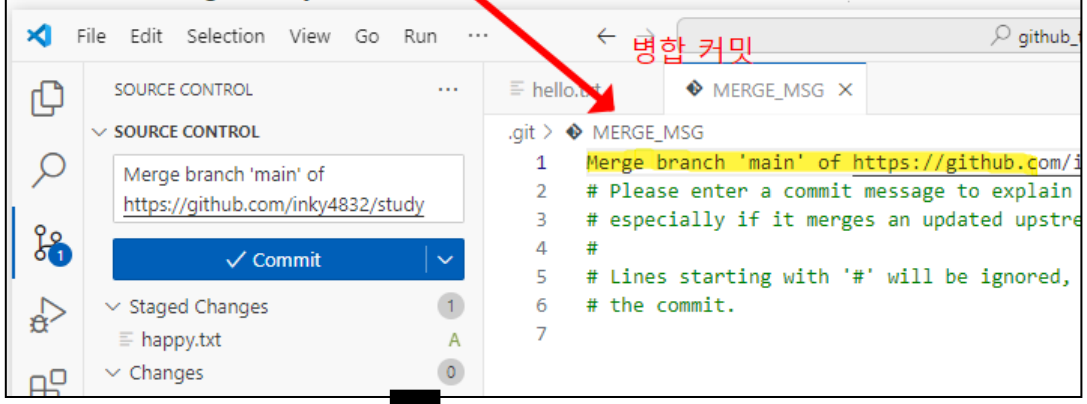
원격 저장소 변경 (happy.txt 파일 추가)



3. pull 이용한 원격 저장소와 로컬 저장소 병합하기

git pull 명령어는 원격 저장소의 커밋 내역을 로컬로 가져와서 자동으로 병합한다.

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (main)
$ git pull origin main
From https://github.com/inky4832/study
 * branch            main       -> FETCH_HEAD
hint: Waiting for your editor to close the file...
```

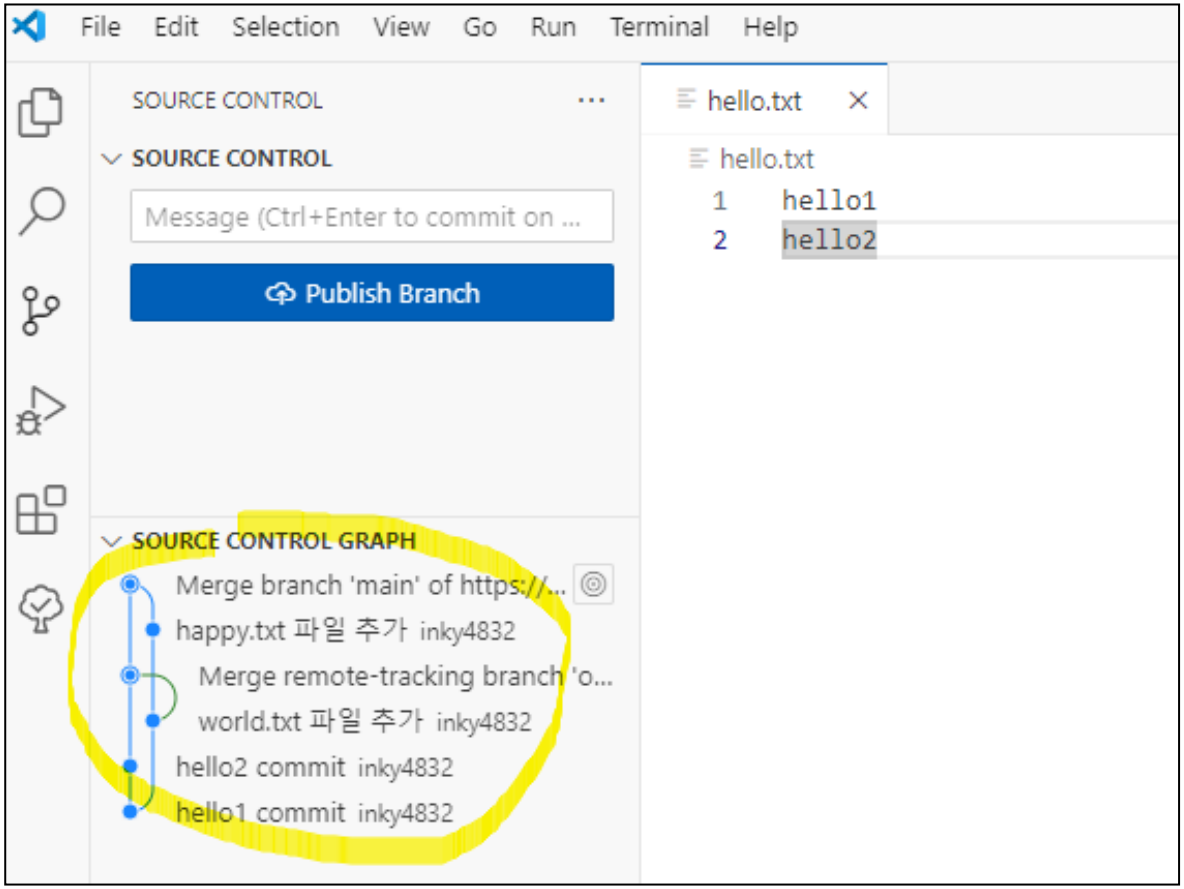



```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (main)
$ git pull origin main
From https://github.com/inky4832/study
 * branch            main       -> FETCH_HEAD
Merge made by the 'ort' strategy.
 happy.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 happy.txt
```

병합 커밋

```
inky4@DESKTOP-KERHMJL MINGW64 ~/github_tutorial (main)
$ git log --oneline
43c28ed (HEAD -> main) Merge branch 'main' of https://g
af16fc7 (origin/main) happy.txt 파일 추가
f44b872 Merge remote-tracking branch 'origin/main'
dc73bc0 world.txt 파일 추가
a2ad850 hello2 commit
05086a0 hello1 commit
```

4. VSC 에서 커밋 graph 보기



clone

1. clone

개요

github에 생성된 원격 저장소를 로컬에 복제하는 방법이다.
clone 하면 로컬 컴퓨터에 새로운 디렉터리가 생성되고 초기화까지 된다.(git init)
따라서 로컬의 git저장소에서 작업하면 안되고 그냥 일반 디렉터리에서 작업한다.
git clone 명령어는 git 명령어이기 때문에 github에 국한되지 않는다. (gitlab 가능)

문법: git clone https://~ [로컬저장소명]

[로컬저장소명]은 생략가능. 생략하면 원격 저장소명으로 로컬저장소가 생김.

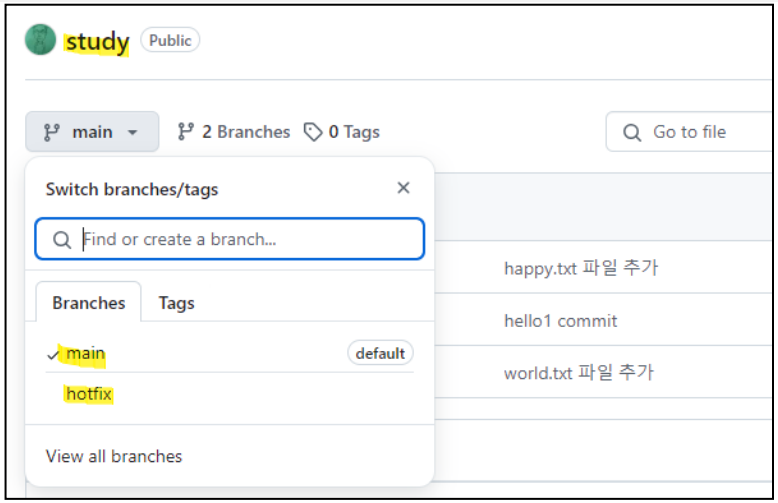
예> git clone https://github.com/inky4832/study.git [git-demo]

특징

만약 원격에 main 브랜치를 제외한 여러 개의 브랜치가 있다고 가정했을 때 clone 하더라도 모든 원격 브랜치를 가져오지 않는다.
main 브랜치만 자동으로 추적하고 나머지는 checkout + switch 해야 로컬에 브랜치가 생성된다.

2. clone 실습

가. 원격 저장소에 main 과 hotfix 브랜치 생성



나. 로컬 디렉터리에서 git clone 실행

```
inky4@DESKTOP-KERHMJL MINGW64 /c/temp
$ pwd
/c/temp

inky4@DESKTOP-KERHMJL MINGW64 /c/temp
$ ls

inky4@DESKTOP-KERHMJL MINGW64 /c/temp
$ git clone https://github.com/inky4832/study.git
Cloning into 'study'...
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 9 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (9/9), done.
```

```
inky4@DESKTOP-KERHMJL MINGW64 /c/temp
$ cd study/   clone 하면 자동으로 디렉터리 생성됨

inky4@DESKTOP-KERHMJL MINGW64 /c/temp/study (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

inky4@DESKTOP-KERHMJL MINGW64 /c/temp/study (main)
$ ls -al
total 11
drwxr-xr-x 1 inky4 197609 0 Sep 22 19:32 ./
drwxr-xr-x 1 inky4 197609 0 Sep 22 19:32 ../
drwxr-xr-x 1 inky4 197609 0 Sep 22 19:32 .git/
-rw-r--r-- 1 inky4 197609 7 Sep 22 19:32 happy.txt
-rw-r--r-- 1 inky4 197609 6 Sep 22 19:32 hello.txt
-rw-r--r-- 1 inky4 197609 7 Sep 22 19:32 world.txt
```

clone 하면 자동으로 git init 까지 됨

2. clone 실습

다. clone 된 브랜치 확인

```
inky4@DESKTOP-KERHMJL MINGW64 /c/temp/study (main)
$ git branch
* main
inky4@DESKTOP-KERHMJL MINGW64 /c/temp/study (main)
$ git branch -r
  origin/HEAD -> origin/main
  origin/hotfix
  origin/main
```

로컬에 hotfix 브랜치 정보가 없음

main 브랜치만 자동으로 추적됨

원격에는 hotfix 브랜치 정보가 있음

라. 누락된 hotfix 브랜치 추적

```
inky4@DESKTOP-KERHMJL MINGW64 /c/temp/study (main)
$ git checkout origin/hotfix
Note: switching to 'origin/hotfix'.

You are in 'detached HEAD' state. You can look around,
changes and commit them, and you can discard any commit
state without impacting any branches by switching back
```

```
inky4@DESKTOP-KERHMJL MINGW64 /c/temp/study ((af16fc7...))
$ git switch hotfix
Switched to a new branch 'hotfix'
branch 'hotfix' set up to track 'origin/hotfix'.

inky4@DESKTOP-KERHMJL MINGW64 /c/temp/study (hotfix)
$ git branch
* hotfix
  main
```

로컬에 hotfix 브랜치가 생성됨

git 협업 워크 플로우

1. 협업 워크플로우

개요

프로젝트 구축할 때 협업 개발자와 함께 코드 관리를 github을 활용하여 관리하는 방법.

구축 방법1

중앙집중형 워크플로우로서 모든 협업 개발자가 단일 브랜치인 main 에서 작업하는 방법으로 협업 워크플로우중에서 가장 최악이다. 절대로 사용하면 안되는 방법이다.

구축 방법2 (*)

중요한 feature 브랜치 워크플로우로서 main 브랜치가 아닌 개별 브랜치인 feature 브랜치에서 기능을 추가하는 방법이다. 가장 일반적인 작업형태이다.

main 브랜치는 프로젝트의 공식적인 이력을 볼 수 있는 브랜치이다.

모든 작업은 별도의 feature 브랜치에서 하고 적합한 때가 오면 feature 브랜치를 main 브랜치와 병합한다. 이렇게 하면 여러 명이 작업해도 main 브런치를 망가뜨릴 위험이 없어진다.

main 브랜치는 무결성이 보장되어야 한다.

각각의 작업은 모두 별도의 feature 브랜치에서 수행되고 필요 없는 브랜치는 버려지거나 필요한 브랜치는 공식적으로 병합되어 이력에 남게 된다. 즉 main 브랜치만 남게 된다.

2. Feature 브랜치 워크플로우 병합

PR (Pull Request)

누군가는 feature 브랜치에 반영된 변경사항을 main 브랜치와 병합해야 된다.

이때 누구든지 모두가 병합하도록 하는 것 보다는 특히 조직에서 절대적으로 코드를 무결하게 유지해야 하거나 개발한 S/W 사용자를 고려해야 할 수도 있는데, 누군가가 엉망인 코드를 main 브랜치에 커밋했을 때 큰일이 발생될 수 있다.

결국 우리나 다른 누군가가 병합하더라도 병합을 해도 되는지 확인 하고 승인 받는 절차가 필요하다. 누군가의 요청으로 병합을 하게 됐는지 상관없이 PR를 이용하면 코드 리뷰 단계를 거치게 된다.

즉 PR은 개발자가 새로운 작업을 브랜치에 올렸을 때 팀원이나 공동 작업자들에게 검토를 요청하고 병합을 승인 또는 반려해달라고 요청하는 방법이다.

3. Feature 브랜치 워크플로우 실습

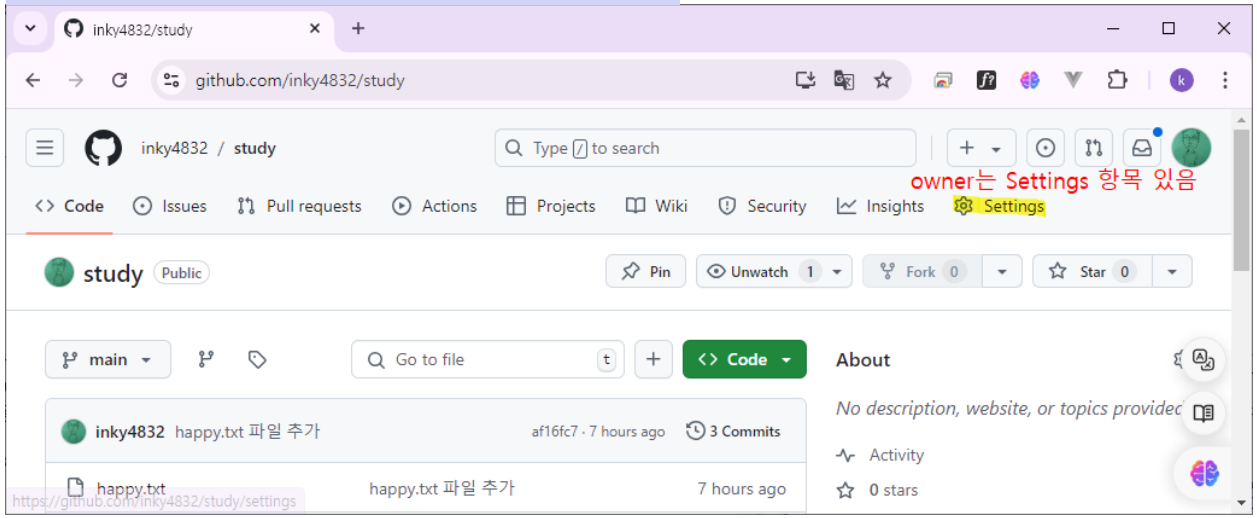
시나리오

Github의 study 원격 저장소의 owner 인 inky4832@daum.net 가 협업자로 inky4833@gmail.com 을 등록하여 프로젝트를 구축하는 실습이다.

1) owner 인 inky4832가 github 에서 inky4833 을 협력자로 등록

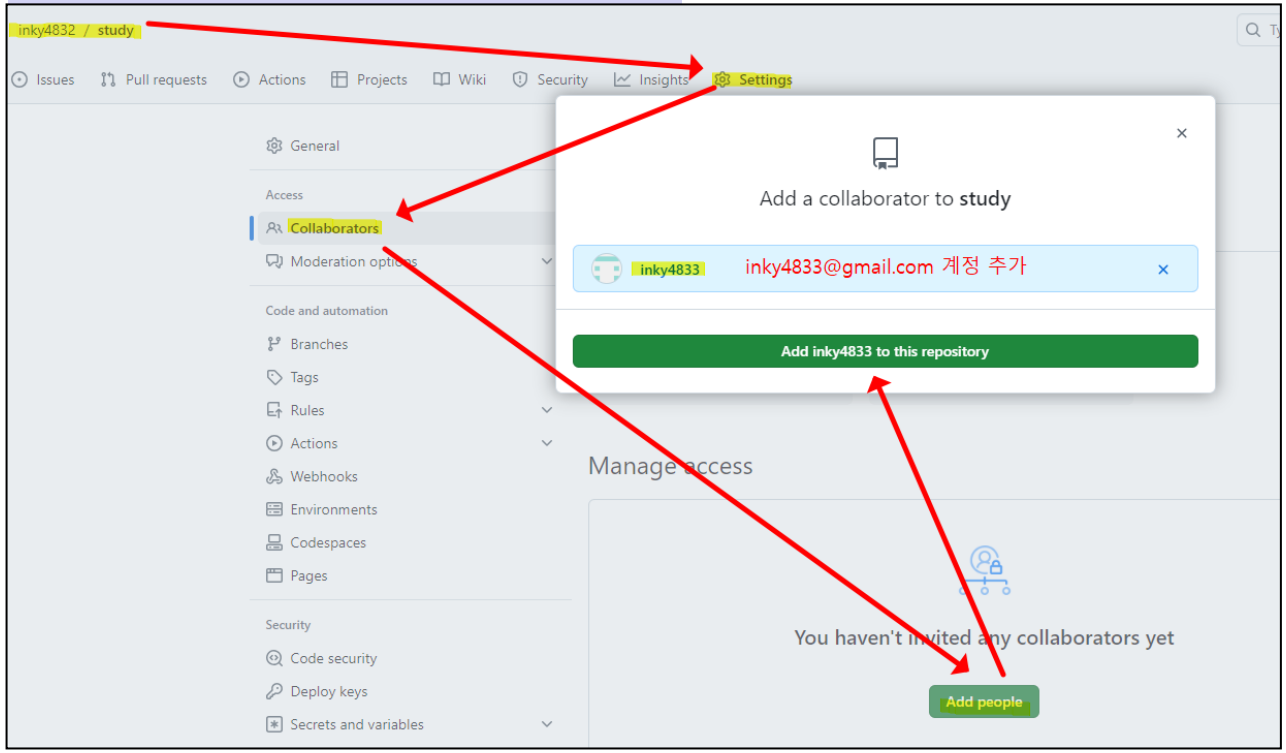
study 저장소 > Settings > Collaborators > Add people 버튼 > inky4833 github 계정 추가 > inky4833@gmail.com 으로 메일 전송됨 > 메일 확인 후 View invitation 버튼 클릭 > 협력자 github 에서 Accept invitation 버튼 클릭 > 협력자 github에 study 저장소가 생성됨. (Settings 링크 없음)

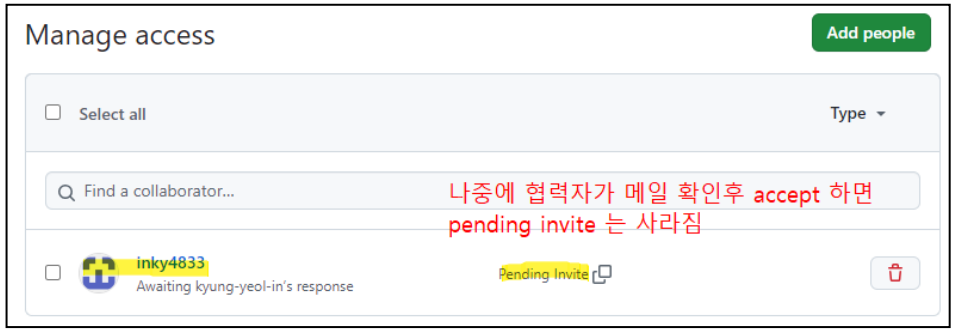
inky4832 owner github 화면



3. Feature 브랜치 워크플로우 실습

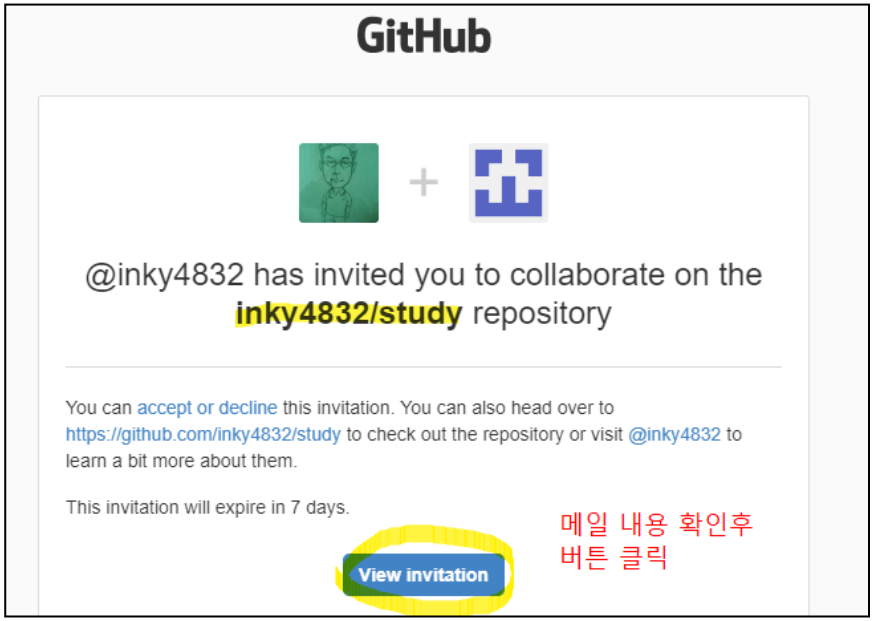
inky4832 owner github 화면



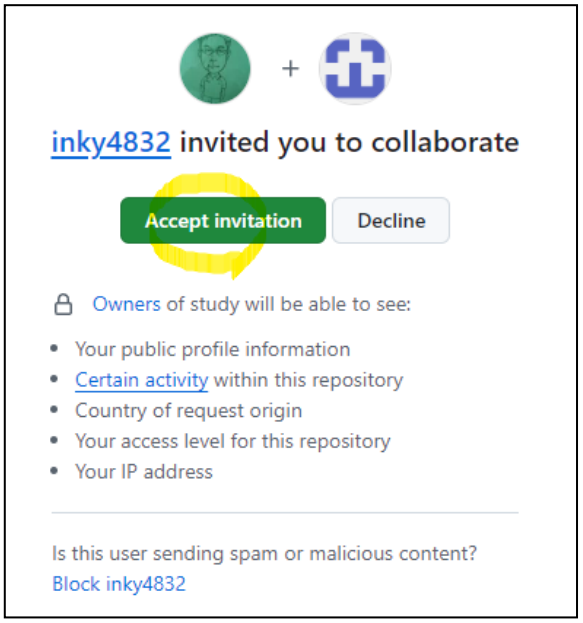


3. Feature 브랜치 워크플로우 실습

inky4833 협력자 이메일 화면

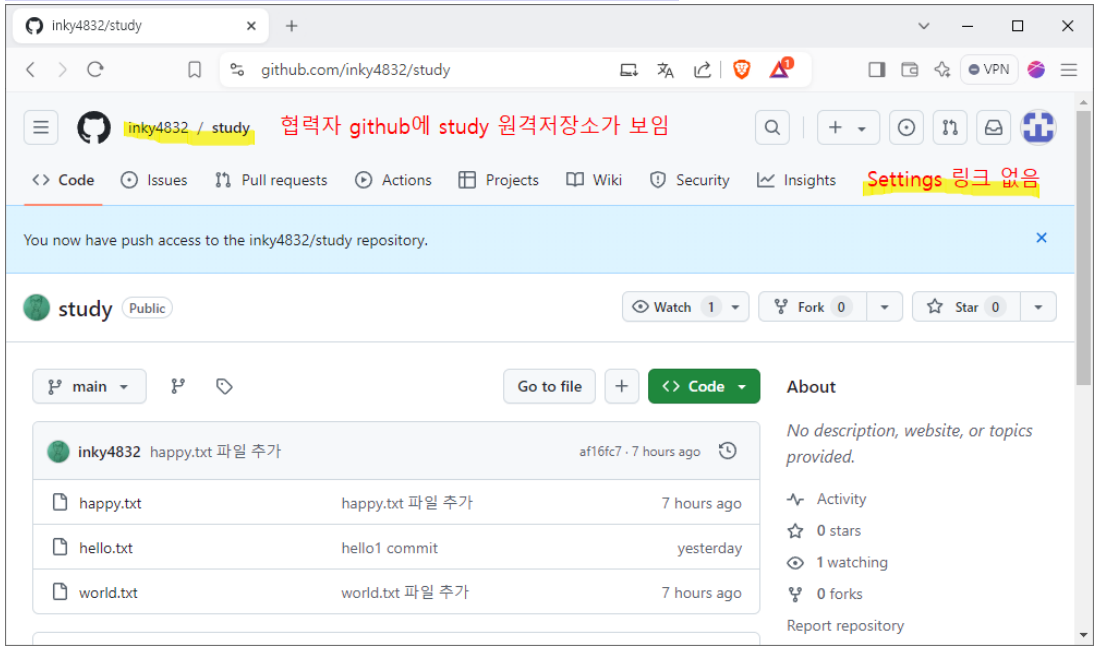


inky4833 협력자 github 화면

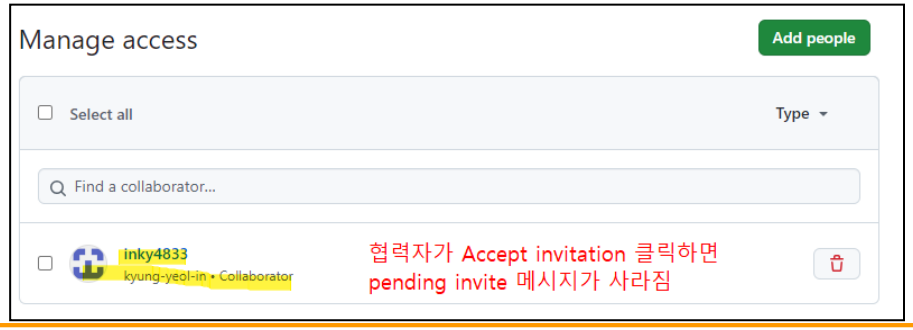


3. Feature 브랜치 워크플로우 실습

inky4833 협력자 github 화면



inky4832 owner github 화면



3. Feature 브랜치 워크플로우 실습

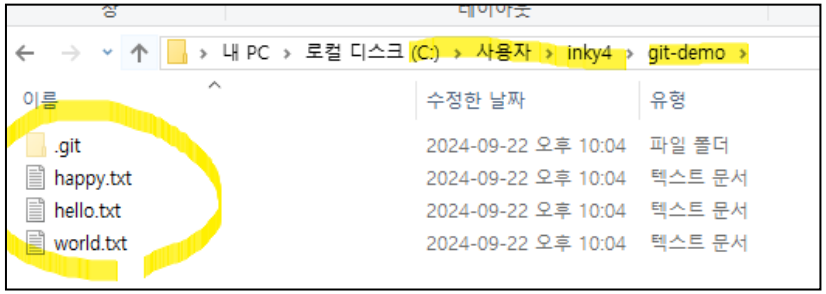
2) 협력자 사용자와 이메일로 변경

```
git config --global user.name "inky4833"
git config --global user.email "inky4833@gmail.com"
```

3) 협력자 PC에서 git clone 으로 inky4832 owner 원격 저장소를 복제

```
inky4@DESKTOP-KERHMJL MINGW64 ~
$ pwd
/c/Users/inky4

inky4@DESKTOP-KERHMJL MINGW64 ~
$ git clone https://github.com/inky4832/study.git git-demo
Cloning into 'git-demo'...
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 9 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (9/9), done.
```

git-demo 디렉터리가 생성되고
git init 까지 실행됨



3. Feature 브랜치 워크플로우 실습

4) 협력자 PC에서 원격 저장소 정보 보기

```
inky4@DESKTOP-KERHMJL MINGW64 ~
$ cd git-demo/

inky4@DESKTOP-KERHMJL MINGW64 ~/git-demo (main)
$ git remote -v
origin  https://github.com/inky4832/study.git (fetch)
origin  https://github.com/inky4832/study.git (push)

inky4@DESKTOP-KERHMJL MINGW64 ~/git-demo (main)
$ git remote get-url origin
https://github.com/inky4832/study.git
```

5) 협력자 PC에서 feature 브랜치 생성후 작업 추가 및 커밋

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git-demo (main)
$ git branch feature        feature 브랜치 생성

inky4@DESKTOP-KERHMJL MINGW64 ~/git-demo (main)
$ git switch feature        feature 브랜치로 이동
Switched to branch 'feature'

inky4@DESKTOP-KERHMJL MINGW64 ~/git-demo (feature)
$ touch study.txt           study.txt 파일 생성
```

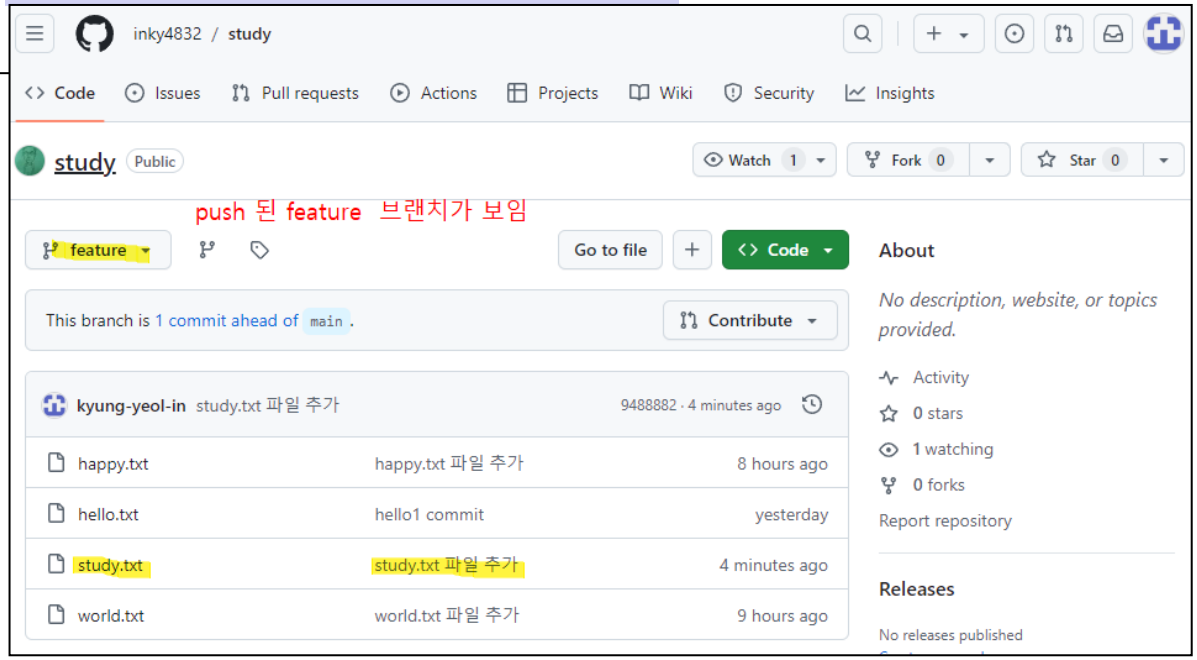
```
inky4@DESKTOP-KERHMJL MINGW64 ~/git-demo (feature)
$ git add study.txt

inky4@DESKTOP-KERHMJL MINGW64 ~/git-demo (feature)
$ git commit -m "study.txt 파 일  추 가 "
[feature 9488882] study.txt 파 일  추 가
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 study.txt
```

3. Feature 브랜치 워크플로우 실습

6) 협력자 PC에서 inky4832 owner 원격 저장소에 push

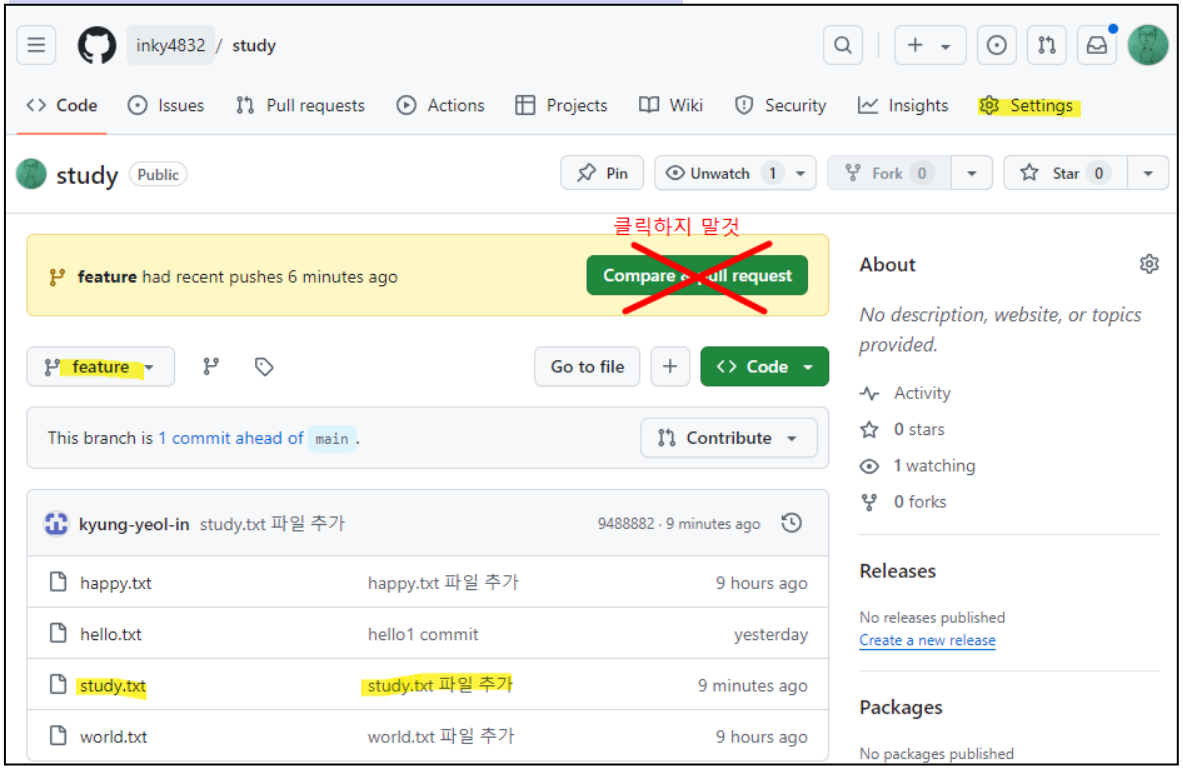
```
inky4@DESKTOP-KERHMJL MINGW64 ~/git-demo (feature)
$ git push origin feature
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 351 bytes | 351.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature' on GitHub by visiting:
remote:      https://github.com/inky4832/s
remote:
To https://github.com/inky4832/study.git
 * [new branch]      feature -> feature
```

inky4833 협력자 github 화면



3. Feature 브랜치 워크플로우 실습

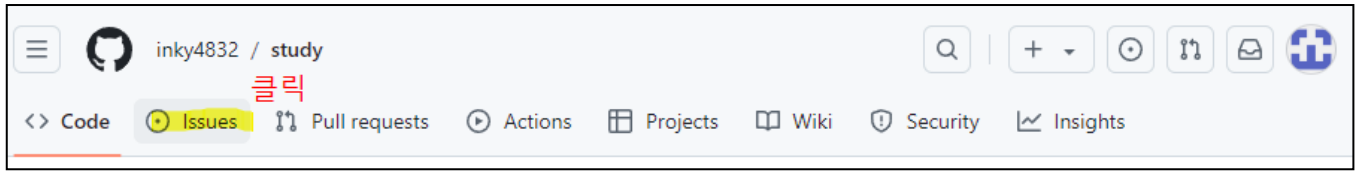
inky4832 owner github 화면

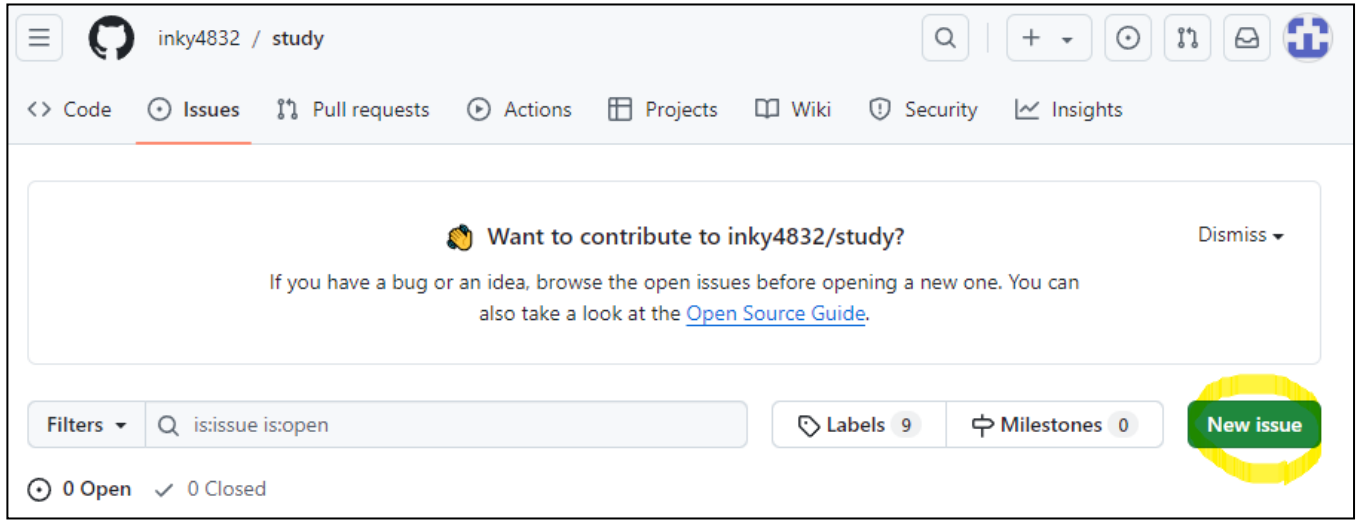


3. Feature 브랜치 워크플로우 실습

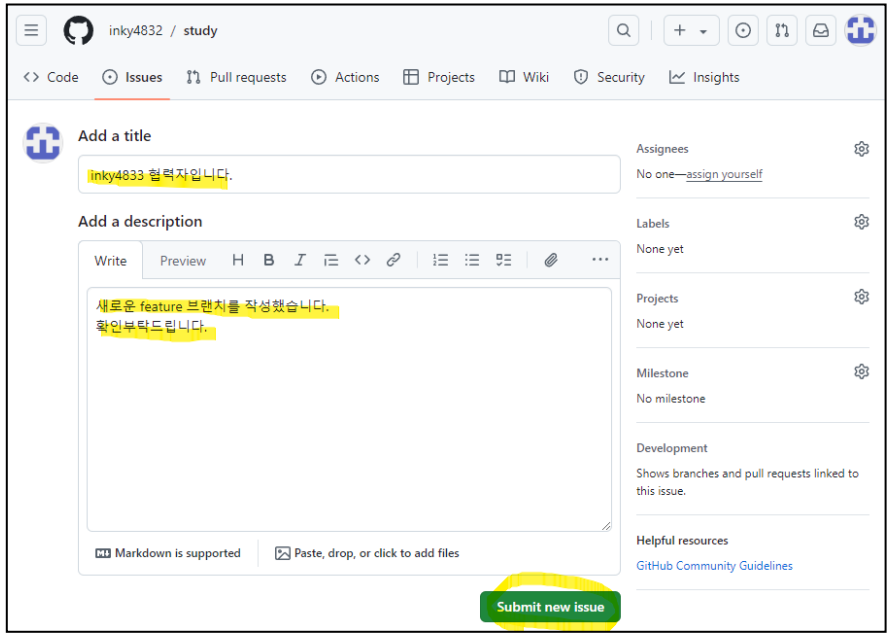
7) 협력자 github에서 특정 이슈를 inky4832 owner에게 보내기 (옵션 기능)

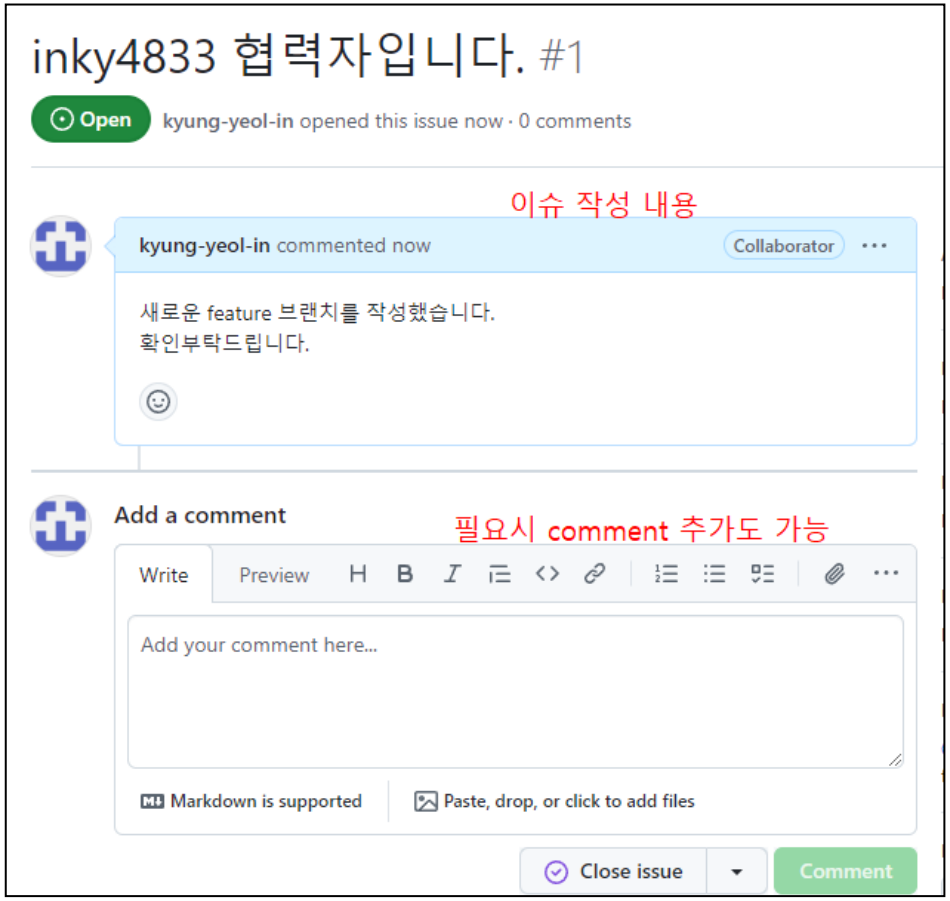
inky4833 협력자 github 화면





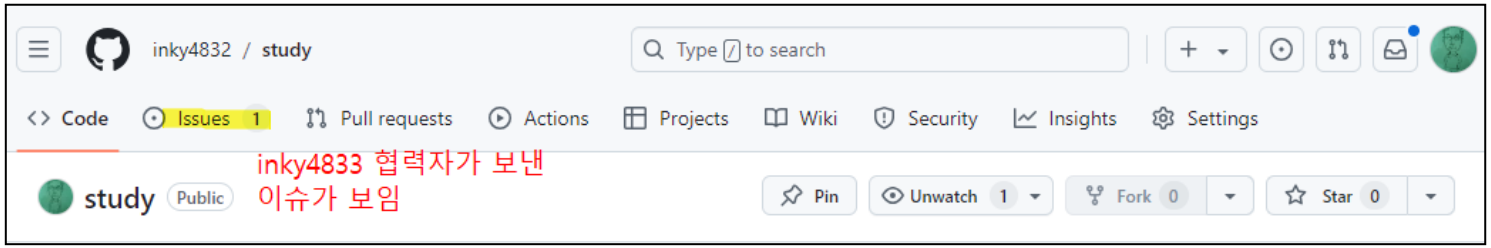
3. Feature 브랜치 워크플로우 실습

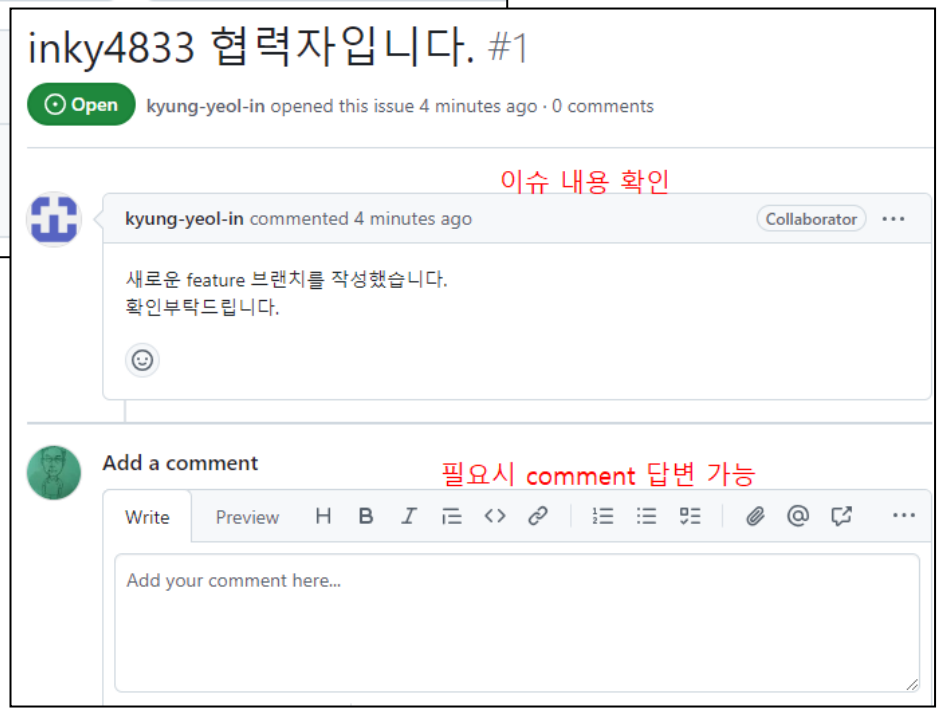




3. Feature 브랜치 워크플로우 실습

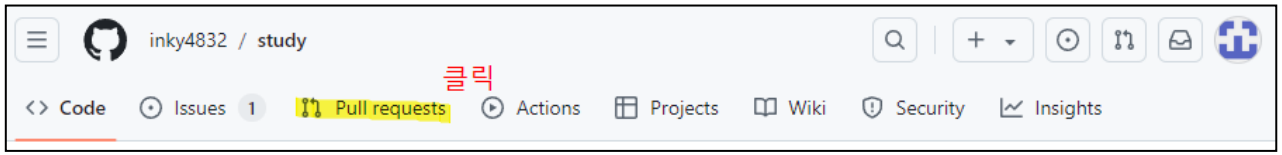
inky4832 owner github 화면

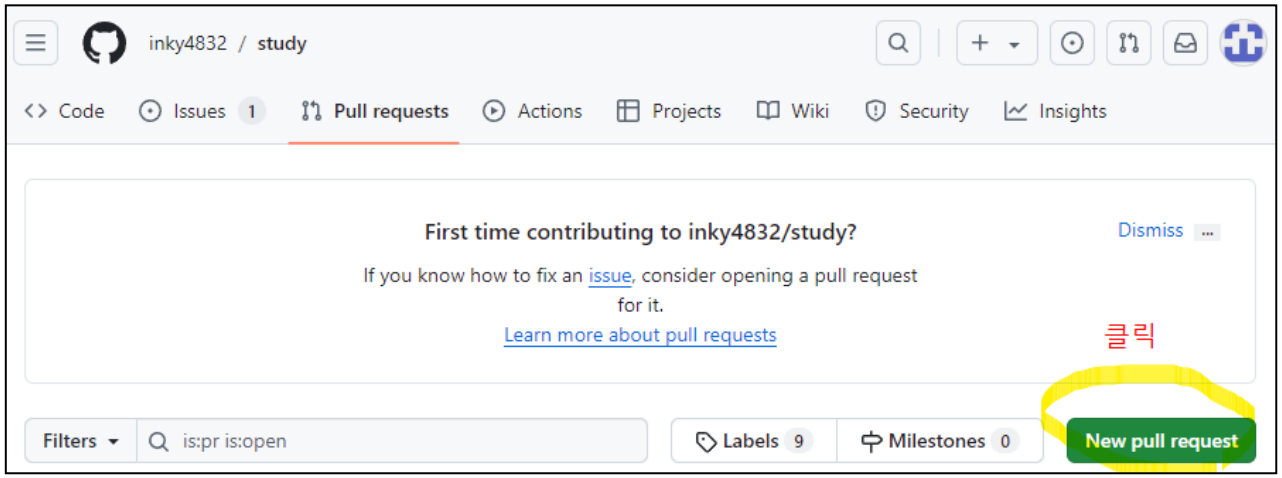




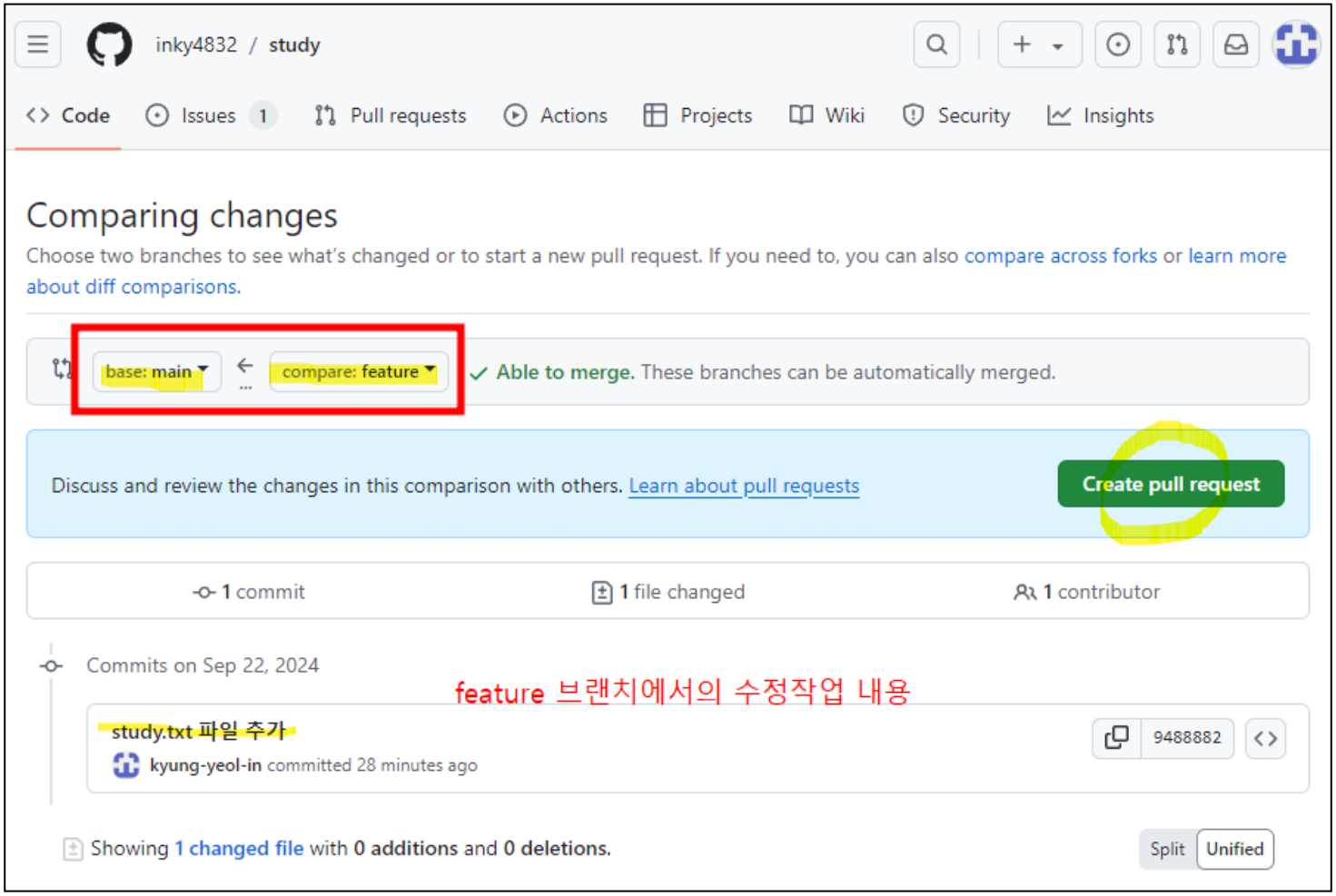
8) 협력자 github에서 feature 브랜치를 병합하도록 inky4832 owner에게 PR 요청하기

inky4833 협력자 github 화면

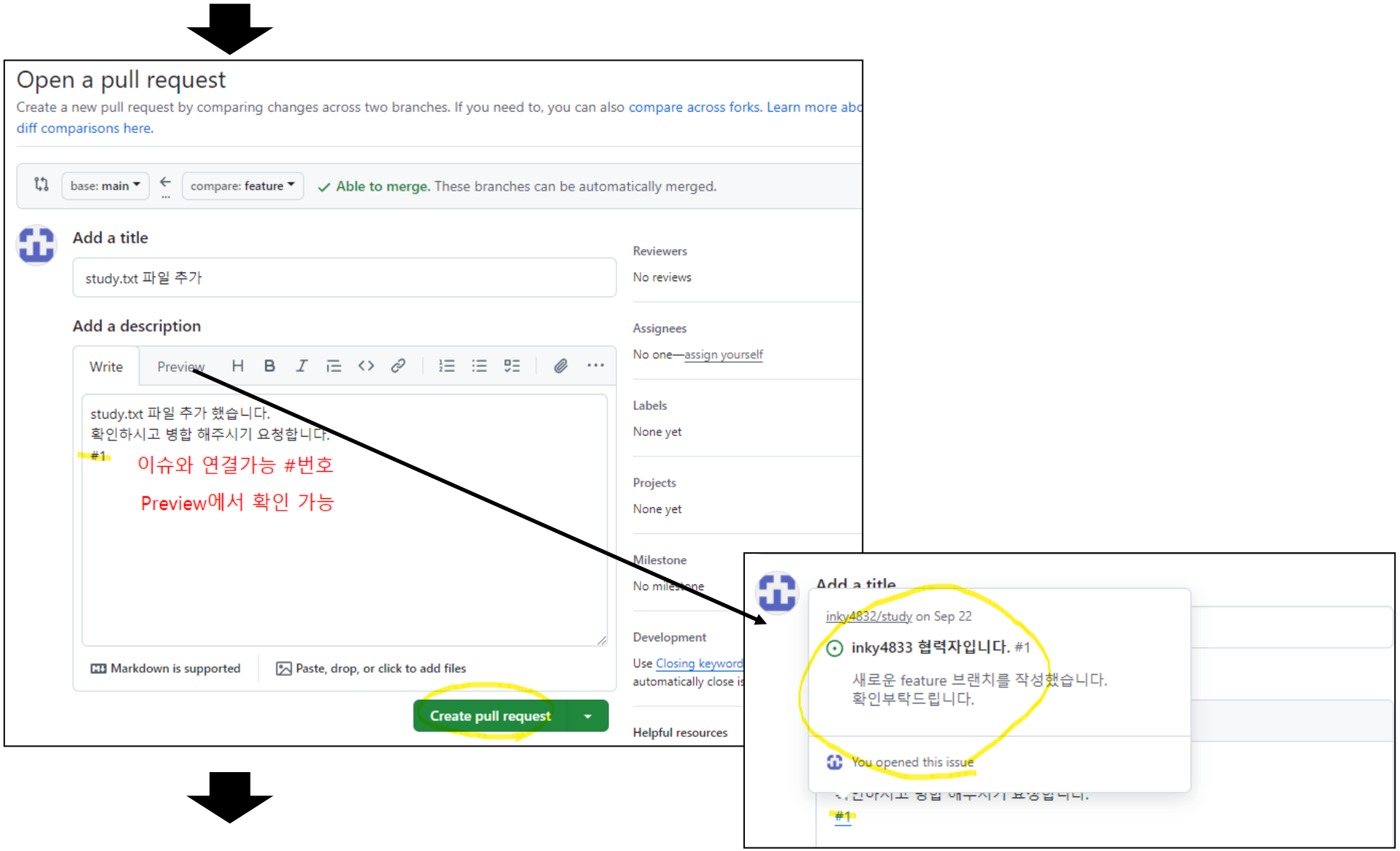




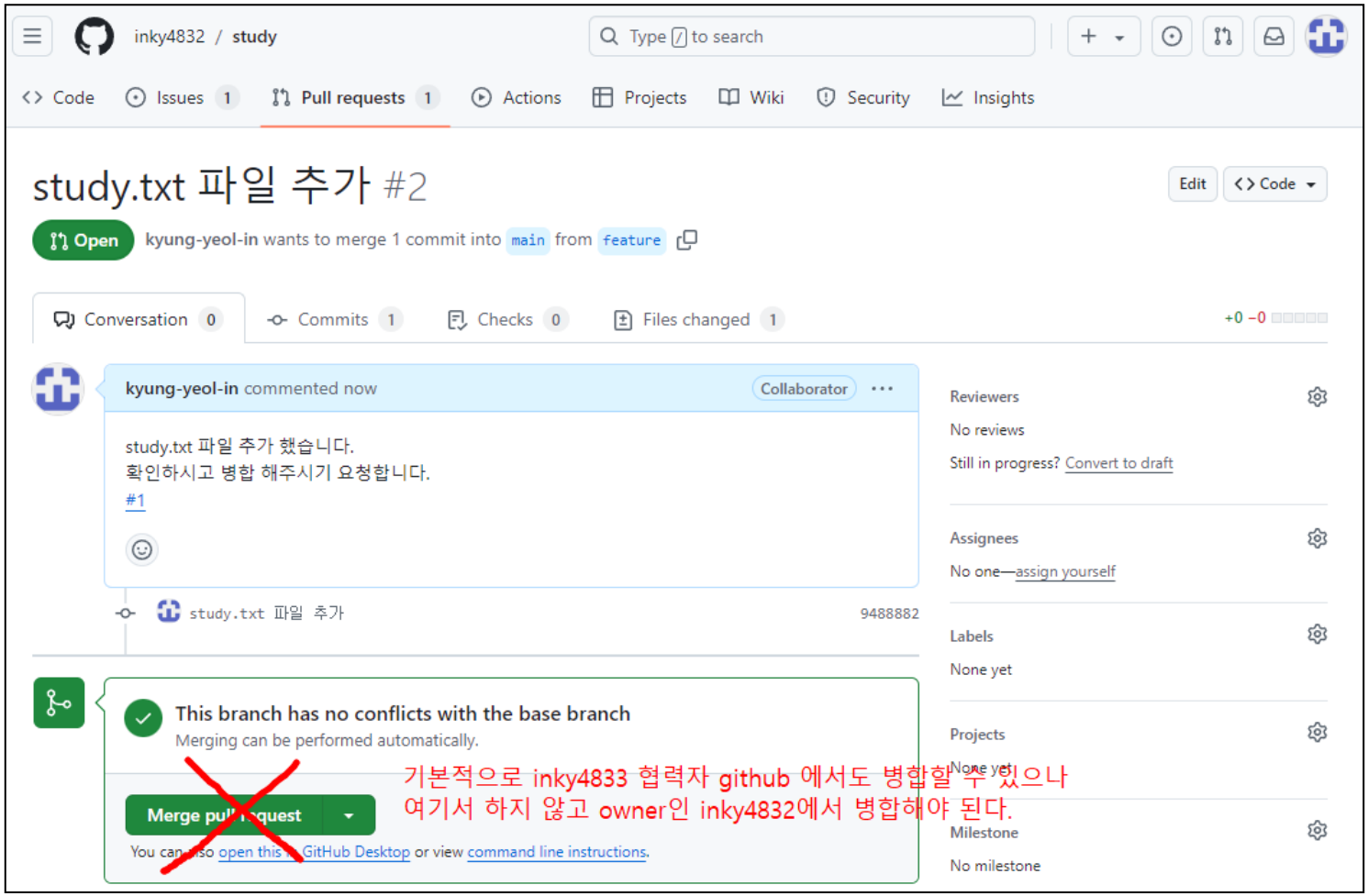
3. Feature 브랜치 워크플로우 실습



3. Feature 브랜치 워크플로우 실습



3. Feature 브랜치 워크플로우 실습

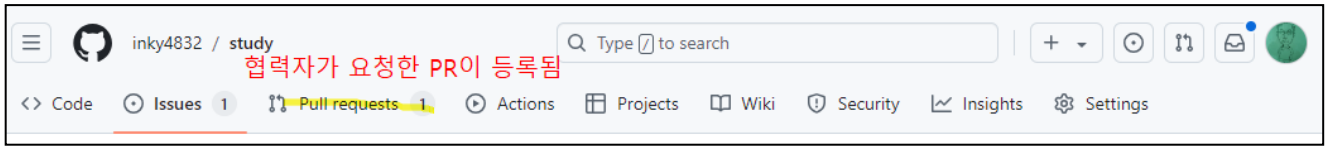


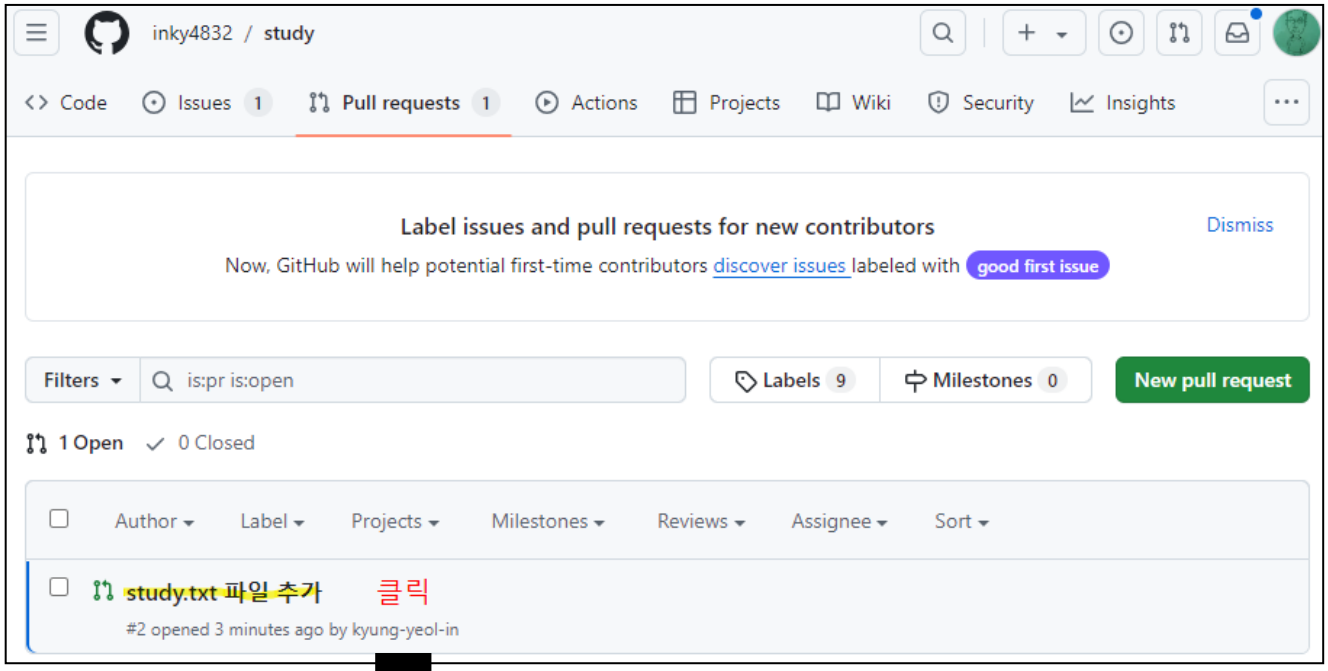
기본적으로 inky4833 협력자 github 에서도 병합할 수 있으나
여기서 하지 않고 owner인 inky4832에서 병합해야 된다.

3. Feature 브랜치 워크플로우 실습

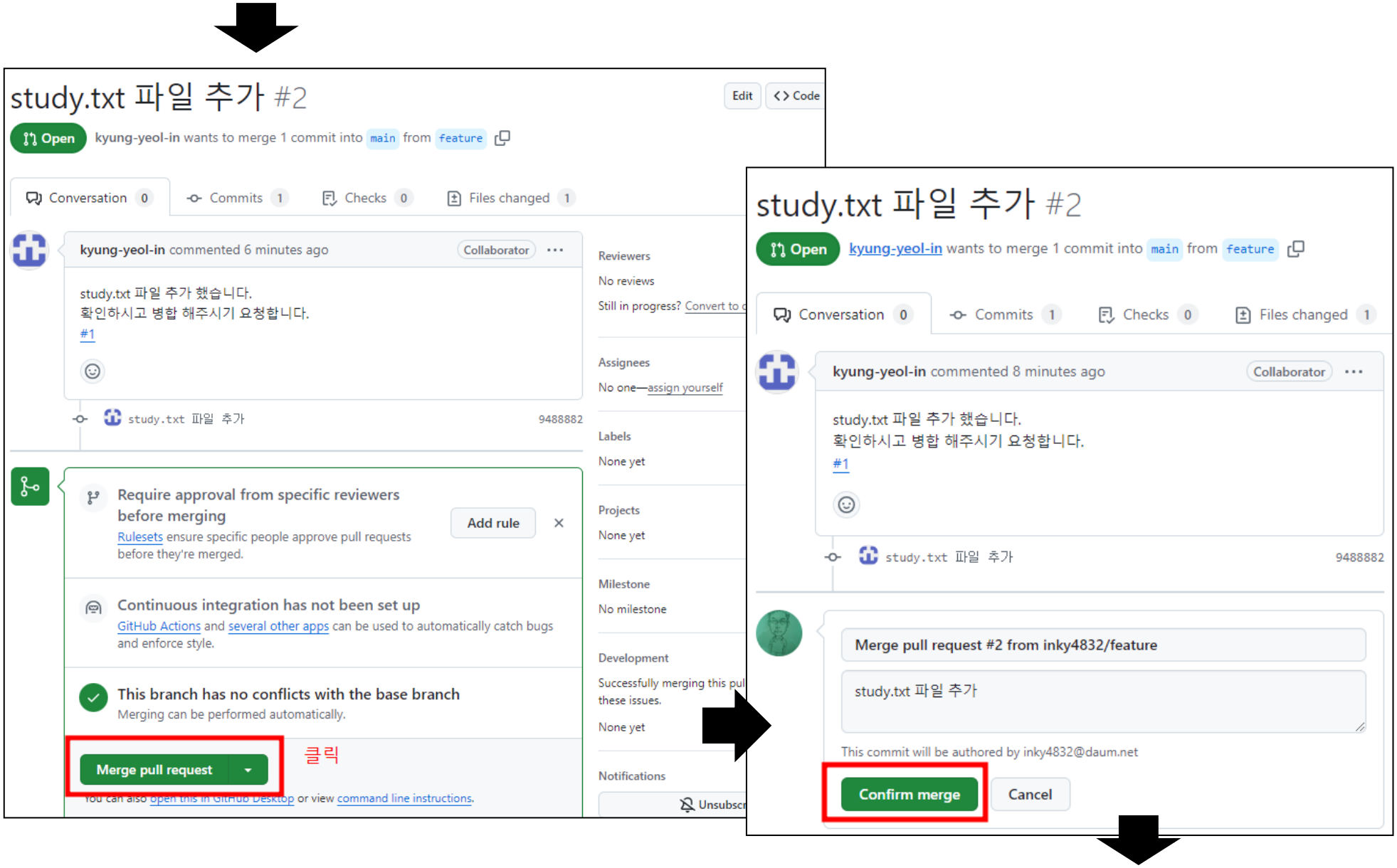
9) inky4832 owner github 에서 PR 요청 처리하기

inky4832 owner github 화면

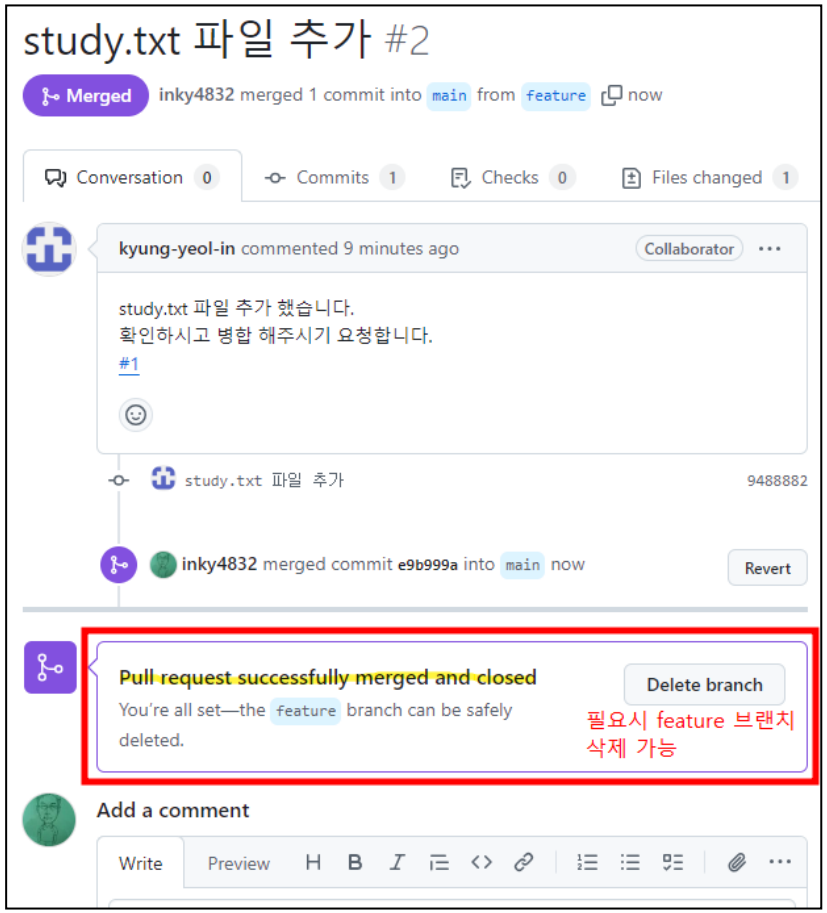




3. Feature 브랜치 워크플로우 실습



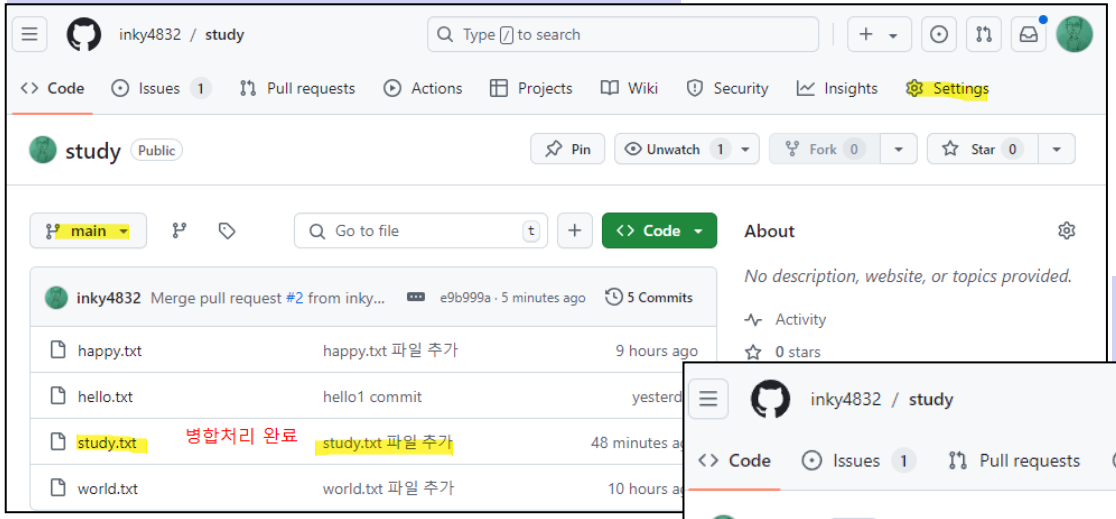
3. Feature 브랜치 워크플로우 실습



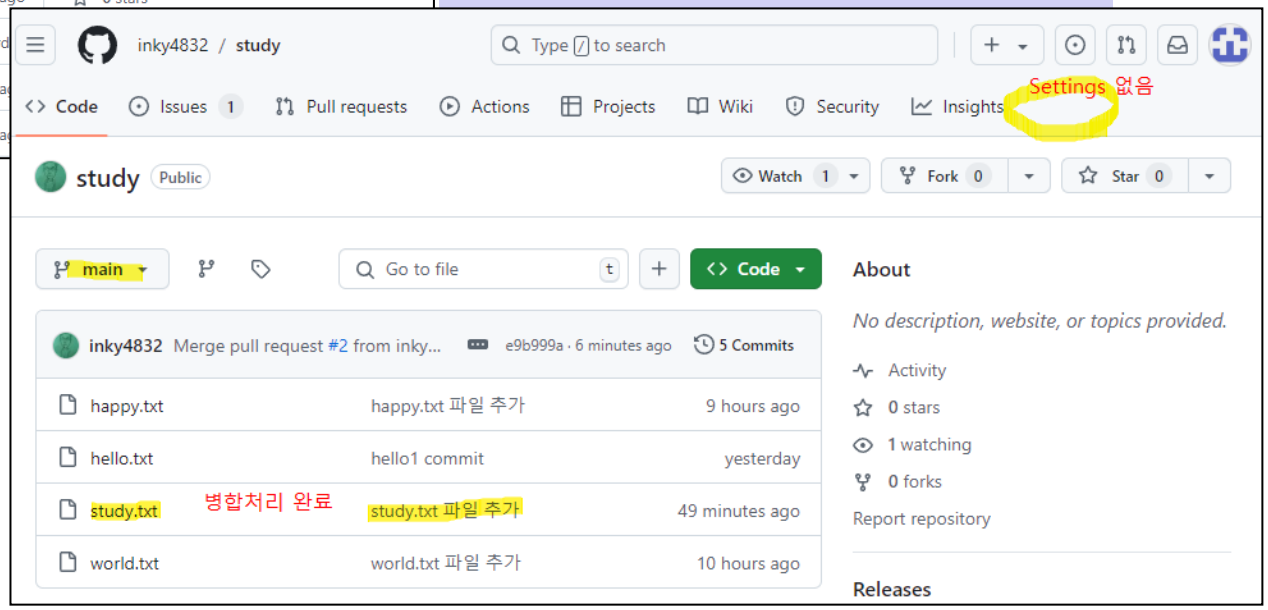
3. Feature 브랜치 워크플로우 실습

10) inky4832 owner 및 inky4833 협력자 github 에서 PR 처리 확인하기

inky4832 owner github 화면



inky4833 협력자 github 화면



11) 원격 저장소 최신 커밋을 로컬 저장소에 반영

git pull 작업전 로컬 저장소 목록

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git-demo (main)
$ ls -al
total 31
drwxr-xr-x 1 inky4 197609 0 Sep 22 23:05 ./
drwxr-xr-x 1 inky4 197609 0 Sep 22 22:04 ../
drwxr-xr-x 1 inky4 197609 0 Sep 22 23:05 .git/
-rw-r--r-- 1 inky4 197609 7 Sep 22 22:04 happy.txt
-rw-r--r-- 1 inky4 197609 6 Sep 22 22:04 hello.txt
-rw-r--r-- 1 inky4 197609 7 Sep 22 22:04 world.txt
```

git pull 작업후 로컬 저장소 목록

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git-demo (main)
$ git pull origin main
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack
Unpacking objects: 100% (1/1), 915 bytes | 305.00 K
From https://github.com/inky4832/study
 * branch            main       -> FETCH_HEAD
   af16fc7..e9b999a  main       -> origin/main
Updating af16fc7..e9b999a
Fast-forward
 study.txt | 0
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 study.txt
```

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git-demo (main)
$ ls -al
total 31
drwxr-xr-x 1 inky4 197609 0 Sep 22 23:06 ./
drwxr-xr-x 1 inky4 197609 0 Sep 22 22:04 ../
drwxr-xr-x 1 inky4 197609 0 Sep 22 23:06 .git/
-rw-r--r-- 1 inky4 197609 7 Sep 22 22:04 happy.txt
-rw-r--r-- 1 inky4 197609 6 Sep 22 22:04 hello.txt
-rw-r--r-- 1 inky4 197609 0 Sep 22 23:06 study.txt
-rw-r--r-- 1 inky4 197609 7 Sep 22 22:04 world.txt
```

Thank you